

**ANALISIS OPTIMASI REAL-TIME PIPELINE NVIDIA
DEEPSTREAM UNTUK APLIKASI ADAS BERBASIS
EDGE DEVICE**

(Naskah gabungan Bab I sampai IV, draf untuk diperiksa dosen pembimbing)

SKRIPSI

Disusun dan diajukan oleh

PERDI

D121221015

DEPARTEMEN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
2026

DAFTAR ISI

(Klik kanan pada area di bawah ini di Microsoft Word, lalu pilih “Update Field” untuk menampilkan daftar isi otomatis.)

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem bantuan pengemudi atau *advanced driver assistance systems* (ADAS) menunjukkan bahwa teknologi ini semakin diposisikan sebagai komponen penting dalam peningkatan keselamatan berkendara. Temuan terbaru menunjukkan bahwa penerapan ADAS berpotensi menurunkan angka kecelakaan berat serta mendukung agenda keselamatan jalan raya secara lebih luas, khususnya ketika fitur-fitur bantu berkendara mulai diadopsi secara lebih merata pada kendaraan modern (Costa et al., 2025). Sejalan dengan itu, kajian terhadap ADAS juga menunjukkan bahwa pengguna cenderung menilai sistem ini mampu meningkatkan rasa aman dan kenyamanan berkendara, meskipun masih dibutuhkan penyempurnaan berkelanjutan agar manfaatnya benar-benar optimal dalam konteks keselamatan (Neumann, 2024). Kondisi tersebut menegaskan bahwa kebutuhan akan sistem bantuan pengemudi yang andal bukan lagi sekadar pilihan teknologi, melainkan kebutuhan nyata dalam upaya menekan risiko kecelakaan di jalan.

Dalam konteks kendaraan cerdas, kemampuan sistem untuk memahami lingkungan secara cepat dan akurat menjadi prasyarat utama. ADAS sangat bergantung pada pemrosesan video dan citra secara terus-menerus agar kendaraan dapat mengenali objek penting, membaca situasi lalu lintas, serta merespons potensi bahaya secara tepat waktu. Studi terkini menekankan bahwa penguatan ADAS melalui *computer vision* dan pendekatan *machine learning* memang meningkatkan kapabilitas deteksi, tetapi sekaligus menuntut proses komputasi yang lebih besar dan berkelanjutan (Shah et al., 2025). Pada saat yang sama, implementasi berbasis perangkat *embedded edge* untuk deteksi objek dan pembentukan informasi lingkungan menunjukkan bahwa pengolahan data visual secara *real-time* merupakan aspek mendasar dalam sistem kendaraan yang aman dan responsif (Choi et al., 2024).

Oleh karena itu, *real-time processing* bukan hanya karakteristik teknis, melainkan inti dari keandalan sistem keselamatan pada kendaraan cerdas.

Persoalan utama muncul ketika kebutuhan pemrosesan *real-time* berhadapan dengan tuntutan latensi yang sangat rendah. Pada sistem berbasis *edge*, keterlambatan sekecil apa pun dapat mengurangi kualitas keputusan dan menurunkan efektivitas peringatan dini, terutama ketika beban inferensi berjalan secara dinamis di lingkungan jaringan yang berubah-ubah. Kajian mengenai *inference serving* dengan *end-to-end latency SLOs* menunjukkan bahwa ketepatan waktu penyajian inferensi sama pentingnya dengan akurasi model, khususnya pada jaringan *edge* yang bersifat dinamis (Nigade et al., 2024). Di sisi lain, penelitian tentang penempatan server *edge* menegaskan bahwa pengaturan sumber daya yang tidak tepat dapat memperbesar latensi transmisi dan memperberat beban kerja sistem, sehingga mengganggu pencapaian kinerja waktu nyata yang dibutuhkan aplikasi kritis (Zhang et al., 2024). Dengan demikian, tantangan utama pada ADAS bukan hanya menghasilkan keluaran yang benar, tetapi juga memastikan keluaran tersebut hadir dalam batas waktu yang aman untuk pengambilan keputusan berkendara.

Tantangan tersebut menjadi semakin kompleks ketika sistem dijalankan pada perangkat *embedded* dengan sumber daya terbatas. Optimasi *deployment* pada platform *embedded heterogen* diperlukan agar kebutuhan *frame rate*, akurasi, dan efisiensi energi tetap terpenuhi (Ruiz-Barroso et al., 2025). Platform seperti **Jetson Orin Nano** menawarkan kemampuan komputasi yang memadai untuk kelas perangkat *edge* berbiaya rendah, namun tetap berada dalam batasan konsumsi daya, suhu operasi, dan kapasitas pemrosesan yang jauh lebih ketat dibandingkan platform *edge* kelas atas (mis. Jetson AGX Orin) maupun server kelas pusat. Kajian terhadap perangkat *embedded* menunjukkan bahwa pemrosesan video *real-time* selalu menuntut kompromi antara kecepatan, efisiensi energi, dan stabilitas kinerja, terutama ketika sistem harus mempertahankan luaran yang konsisten pada beban kerja yang berubah-ubah (Suder et al., 2023). Dalam konteks aplikasi keselamatan,

kompromi semacam ini menjadi sangat penting karena keterbatasan sumber daya dapat langsung berpengaruh pada keterlambatan deteksi, penurunan laju pemrosesan, dan berkurangnya kemampuan sistem dalam mempertahankan performa secara berkelanjutan.

Selain beban komputasi, sistem ADAS juga menuntut karakteristik *deterministic* dan stabil karena bersifat *safety-critical*. Setiap proses harus selesai sesuai tenggat waktu agar fungsi peringatan, deteksi, dan respons dapat berjalan secara konsisten di bawah kondisi operasional yang berubah cepat. Studi terbaru menunjukkan bahwa penjadwalan tugas pada ADAS memiliki tingkat kepekaan yang tinggi terhadap *deadline*, dan kegagalan memenuhi batas waktu dapat berimplikasi pada kegagalan sistem secara fungsional (Seyfipoor et al., 2026). Hal ini menegaskan bahwa aplikasi keselamatan memerlukan arsitektur yang tidak hanya cepat, tetapi juga stabil, terprediksi, dan efisien dalam mengelola prioritas pemrosesan. Dengan kata lain, keandalan ADAS tidak dapat dilepaskan dari kemampuan sistem untuk menjaga konsistensi waktu proses dalam kondisi beban tinggi, baik beban tersebut berasal dari tahap deteksi objek (inferensi model dan *post-processing* NMS-nya) maupun dari tahap asosiasi objek antar-frame (*tracking*) yang menjaga identitas objek tetap konsisten sepanjang waktu.

Berdasarkan uraian tersebut, terlihat bahwa penelitian mengenai optimasi *real-time pipeline* pada platform *embedded* menjadi relevan dan mendesak untuk dilakukan, khususnya pada konteks ADAS berbasis Jetson Orin Nano. Permasalahan yang dihadapi bukan semata-mata pada kemampuan mendeteksi objek, melainkan pada bagaimana sistem dapat mempertahankan kecepatan, kestabilan, dan efisiensi pemrosesan secara simultan dalam skenario operasi yang menuntut respons segera, pada setiap tahap pipeline, mulai dari inferensi deteksi, *post-processing* (NMS), hingga *tracking*. Oleh karena itu, penelitian ini diarahkan untuk memahami dan menganalisis faktor-faktor yang memengaruhi kinerja *real-time pipeline* agar sistem ADAS dapat memenuhi kebutuhan keselamatan berkendara secara lebih andal.

1.2 Landasan Teori

1.2.1 ADAS dan Perception Layer pada Kendaraan Cerdas

Advanced Driver Assistance Systems (ADAS) adalah kumpulan teknologi sistem elektronik terintegrasi yang dirancang untuk membantu pengemudi dalam proses berkendara dan memarkir kendaraan guna meningkatkan keselamatan dan kenyamanan. Sistem ADAS modern sangat bergantung pada pemahaman lingkungan sekitar secara *real-time* untuk mendeteksi potensi bahaya, seperti keberadaan kendaraan lain, pejalan kaki, atau infrastruktur jalan (Neumann, 2024).

Secara arsitektural, sistem ADAS terdiri dari tiga lapisan utama: *perception layer* (lapisan persepsi), *planning layer* (lapisan perencanaan), dan *control layer* (lapisan kendali) (Shah et al., 2025). Penelitian ini secara khusus berfokus pada *perception layer*, yaitu tahapan di mana data mentah dari sensor (dalam hal ini, citra video dari kamera) diekstraksi menjadi informasi semantik berwujud deteksi dan pelacakan objek. Pada lapisan ini, keandalan sistem tidak hanya diukur dari seberapa akurat objek dikenali, melainkan juga seberapa cepat informasi tersebut dapat disajikan ke lapisan perencanaan, karena keterlambatan sekecil apapun dapat mengurangi waktu respons pengereman darurat atau peringatan dini.

1.2.2 Konsep Edge Computing dan Analisis Video Real-Time

Pemrosesan *perception layer* pada kendaraan otonom dan ADAS umumnya diimplementasikan menggunakan paradigma *edge computing*, di mana komputasi data dilakukan secara lokal di perangkat (*on-device*) yang berada sedekat mungkin dengan sumber data sensor (Choi et al., 2024). Pendekatan ini meniadakan ketergantungan pada koneksi jaringan *cloud* yang memiliki latensi transmisi dinamis dan rentan terputus (Zhang et al., 2024).

Namun, implementasi Edge AI memiliki batasan mendasar terkait anggaran sumber daya (*resource constraints*). Perangkat *embedded edge* (seperti keluarga

NVIDIA Jetson) beroperasi dengan ketersediaan memori, kapabilitas komputasi (CPU/GPU), dan konsumsi daya (*thermal design power*) yang sangat terbatas dibandingkan server pusat (Suder et al., 2023). Oleh karena itu, *real-time video analytics* pada *edge device* selalu menuntut kompromi yang optimal antara akurasi model, efisiensi energi, dan kecepatan pemrosesan. Kinerja *real-time* dalam konteks ini didefinisikan sebagai kemampuan sistem dalam mengeksekusi *pipeline* secara deterministik guna memenuhi tenggat waktu (*deadline*) laju bingkai (misalnya 30 FPS) secara terus-menerus tanpa penumpukan antrean pemrosesan (Seyfipoor et al., 2026).

1.2.3 NVIDIA DeepStream SDK dan Pipeline Inferensi

NVIDIA DeepStream adalah *Software Development Kit* (SDK) yang dibangun di atas kerangka kerja multimedia GStreamer, dirancang khusus untuk membangun *pipeline* analitik video bertenaga kecerdasan buatan (AI) secara *real-time* dengan memanfaatkan akselerasi perangkat keras NVIDIA.

Karakteristik kunci DeepStream yang membedakannya dari *pipeline* konvensional (seperti integrasi manual berbasis OpenCV) adalah penggunaan arsitektur memori *zero-copy* berbasis NVIDIA Memory Manager (NVMM). Pada *pipeline* standar, *frame* gambar sering disalin secara berulang antara memori CPU (*host*) dan GPU (*device*) pada setiap tahapan (*decode*, *pre-processing*, inferensi, *post-processing*). NVMM mengeliminasi hambatan (*bottleneck*) ini dengan menahan *buffer frame* video secara persisten di dalam memori GPU dari awal hingga akhir *pipeline*.

1.2.4 Deteksi Objek Real-Time dan Arsitektur YOLO

You Only Look Once (YOLO) merupakan keluarga arsitektur jaringan saraf tiruan untuk deteksi objek berbasis pendekatan *single-stage detector*. Berbeda dengan metode *two-stage* (seperti Faster R-CNN) yang memerlukan tahapan proposal *region* sebelum klasifikasi, YOLO memprediksi koordinat *bounding box* dan probabilitas

kelas secara bersamaan dalam satu kali eksekusi (*forward pass*) jaringan penuh. Karakteristik ini membuat YOLO sangat ideal untuk pemrosesan *real-time* di lingkungan ADAS (Ayachi et al., 2025; Dhatrika et al., 2025).

Penelitian ini mengevaluasi berbagai generasi kelas ringan (*nano/tiny*) dari YOLO, seperti YOLOv8, YOLOv9, YOLOv10, dan YOLO26. Perbedaan struktural penting di antara generasi ini terletak pada strategi *post-processing*. Generasi awal hingga v9 menghasilkan banyak prediksi *bounding box* redundan di sekitar objek tunggal, sehingga mewajibkan penggunaan filter *Non-Maximum Suppression* (NMS). Sebaliknya, arsitektur yang lebih modern (seperti YOLOv10n dan YOLO26n) telah mengintegrasikan metode optimasi struktural (*consistent dual assignments*) selama pelatihan, menjadikannya model *NMS-free* yang tidak lagi memerlukan *post-processing* heuristik eksternal, sehingga berpotensi menurunkan latensi inferensi.

1.2.5 Post-Processing Deteksi: Bounding Box, IoU, dan NMS

Keluaran mentah dari arsitektur deteksi objek standar (*non-NMS-free*) terdiri dari sekumpulan *bounding box* kandidat. Setiap kandidat direpresentasikan oleh parameter koordinat (x, y, w, h) dan sebuah nilai keyakinan (*confidence score*) yang menyatakan probabilitas keberadaan objek berserta kelasnya.

Untuk mengevaluasi tingkat tumpang-tindih antar kandidat *bounding box*, digunakan metrik *Intersection over Union* (IoU), yang diformulasikan sebagai rasio luas irisan terhadap luas gabungan dari dua *bounding box* (A dan B):

$$\text{IoU} = \text{Area}(A \cap B) / \text{Area}(A \cup B)$$

Non-Maximum Suppression (NMS) adalah algoritma *post-processing* yang menyaring kandidat berlebih tersebut. Proses NMS standar mengurutkan seluruh *bounding box* berdasarkan *confidence score* tertinggi, mempertahankan *bounding box* dengan skor tertinggi, lalu menghapus *bounding box* lain yang memiliki nilai IoU dengan *bounding box* tertinggi melampaui suatu nilai ambang batas (*threshold*) tertentu (misalnya $\text{IoU} > 0,45$).

Implementasi NMS tradisional (seperti *GreedyNMS*) bersifat sekuensial dan umumnya dieksekusi oleh CPU (*host*), yang seringkali menjadi hambatan latensi pada pemrosesan *edge* akibat proses perpindahan data dan interupsi siklus (Chen et al., 2022; Oro et al., 2022). Untuk menanggulangi hal ini, *plugin EfficientNMS_TRT* mengintegrasikan tahapan NMS secara langsung ke dalam graf komputasi TensorRT. Pendekatan ini memungkinkan perbandingan IoU dan penyaringan objek dilakukan secara paralel penuh (*fully parallelized*) di dalam *kernel* GPU, menghilangkan *bottleneck* transfer memori ke CPU dan mempercepat penyajian inferensi (Yang et al., 2025).

1.2.6 Konsep Dasar Multi-Object Tracking (MOT)

Deteksi objek berbasis *frame* tunggal rentan terhadap ketidakstabilan visual, seperti *flicker* akibat oklusi sementara, perubahan pencahayaan, atau kegagalan detektor secara sesaat. *Multi-Object Tracking* (MOT) mengatasi masalah ini dengan mengasosiasikan deteksi pada *frame* saat ini dengan lintasan historis objek dari *frame-frame* sebelumnya, memberikan identitas unik (ID) yang konsisten.

Algoritma MOT paling efisien berakar pada paradigma *Tracking-by-Detection* yang utamanya menggunakan dua komponen matematis:

1. **Kalman Filter:** sebuah algoritma estimasi rekursif yang memprediksi keadaan kinematik objek di masa depan (posisi dan kecepatan) berdasarkan pengukuran beruntun yang mengandung derau (*noise*).
2. **Hungarian Algorithm:** algoritma optimasi kombinatorial untuk memecahkan masalah penugasan (*assignment problem*). Dalam MOT, algoritma ini digunakan untuk mencocokkan (*bipartite matching*) prediksi lokasi Kalman Filter dengan lokasi *bounding box* deteksi baru, biasanya menggunakan matriks jarak berbasis IoU atau jarak Euclidean.

1.2.7 Algoritma Tracking pada DeepStream

DeepStream menyediakan elemen *nvtracker* yang mendukung berbagai profil pelacakan. Penelitian ini mengevaluasi dua algoritma pelacakan dengan pendekatan desain yang mewakili kedua ujung spektrum beban komputasi:

- **NvSORT** (*Simple Online and Realtime Tracking*): pelacak berbasis *motion-only* murni yang hanya mengandalkan Kalman Filter dan Hungarian Algorithm pada properti koordinat spasial tanpa mengekstraksi atau memproses data piksel gambar sama sekali. Hal ini membuatnya beroperasi sangat cepat dan efisien terhadap utilisasi CPU/GPU, dengan kompromi berkurangnya kemampuan membedakan objek saat terjadi oklusi silang yang kompleks (Shin & Li, 2023).
- **NvDCF** (*Discriminative Correlation Filter*): pelacak hibrida *feature-based* yang mempelajari ciri visual khusus (*learned visual features*) dari setiap target secara *real-time*. NvDCF memproses area piksel dalam *bounding box* untuk membuat filter korelasi, yang memungkinkannya melacak objek saat detektor gagal atau tertutup sebagian secara tangguh. Konsekuensinya, NvDCF membutuhkan beban komputasi yang jauh lebih berat karena mengeksekusi ekstraksi fitur pada setiap objek di setiap *frame*.

1.2.8 Akselerasi Inferensi: TensorRT dan Kuantisasi Presisi

NVIDIA TensorRT adalah *runtime* optimasi *deep learning* yang mengkompilasi model jaringan saraf (seperti arsitektur ONNX) menjadi *engine* yang dieksekusi secara spesifik untuk mikroarsitektur perangkat keras target (GPU Ampere). TensorRT melakukan fusi lapisan (*layer fusion*), kalibrasi presisi, dan pemilihan *kernel* (*kernel auto-tuning*).

Kuantisasi adalah teknik mereduksi kedalaman *bit* yang digunakan untuk merepresentasikan bobot dan aktivasi jaringan. Sementara pelatihan model konvensional dilakukan dalam presisi *Floating-Point* 32-bit (FP32), inferensi pada perangkat *edge* dioptimalkan menggunakan presisi lebih rendah. Presisi *Floating-*

Point 16-bit (FP16) dapat mempercepat kalkulasi berkat penggunaan unit *Tensor Core* pada arsitektur GPU dengan penurunan akurasi yang marginal.

1.2.9 Metrik Evaluasi Kinerja Real-Time Pipeline

Evaluasi kinerja *pipeline* pada komputasi lokal *edge* didasarkan pada efisiensi komputasi, kelancaran eksekusi, serta mempertahankan akurasi dasar (Nigade et al., 2024). Metrik yang digunakan meliputi:

A. Throughput (FPS)

Laju bingkai per detik (*Frames Per Second*) mengukur agregat volume pemrosesan. Diformulasikan secara konseptual dari total latensi *end-to-end* rerata sistem per *frame*:

$$\text{FPS} = 1 / (\text{rerata } t_{e2e} \text{ dalam detik})$$

di mana sistem dikategorikan *real-time* jika nilai laju pemrosesan stabil pada target aplikasi kritis (misalnya minimum 30 FPS).

B. Latensi dan Persentil (P95) Latensi

Latensi adalah durasi tempuh pemrosesan. Pada penelitian ini, dievaluasi latensi *end-to-end* yang diukur dari titik bingkai masuk pada tahap *multiplexing* aliran video hingga keluar di blok antarmuka layar, dan dirincikan secara per-komponen (mis. tahap inferensi model dan tahap *tracking* objek). Dalam sistem *safety-critical* seperti ADAS, rerata (*mean*) rentan menyembunyikan efek variabilitas ekstrem (*jitter*). Oleh karena itu, metrik menggunakan persentil ke-95 (P95) yang mengindikasikan bahwa 95% dari seluruh siklus *frame* diselesaikan dalam durasi kurang dari atau sama dengan nilai latensi tersebut.

C. Akurasi Deteksi

Untuk memastikan optimasi presisi komputasi (FP16) dan implementasi NMS tidak merusak fungsionalitas pengenalan (*sanity-check*), akurasi diukur berdasar matriks kebingungan (*confusion matrix*):

- **Precision (P)**: proporsi deteksi yang relevan (benar): $P = TP / (TP + FP)$
- **Recall (R)**: proporsi target asli yang berhasil ditemukan: $R = TP / (TP + FN)$
- **mean Average Precision (mAP)**: luas area di bawah kurva *Precision-Recall* rerata untuk seluruh kelas objek, dihitung melintasi batas ambang batas (IoU = 0,5 untuk mAP50, dan rentang IoU = 0,5–0,95 untuk mAP50-95).

1.3 Rumusan Masalah

Berdasarkan latar belakang penelitian, rumusan masalah dalam penelitian ini adalah sebagai berikut:

3. Bagaimana kinerja *real-time pipeline* Nvidia DeepStream pada proses deteksi kendaraan di Jetson Orin Nano?
4. Sejauh mana penerapan NMS paralel berbasis TensorRT plugin (EfficientNMS) dapat meningkatkan efisiensi *pipeline* deteksi kendaraan dibandingkan konfigurasi standar?
5. Sejauh mana pemilihan algoritma *tracking* (NvDCF vs. NvSORT) memengaruhi efisiensi komputasi *real-time pipeline* dibandingkan konfigurasi *baseline*?

1.4 Tujuan

Penelitian ini memiliki tujuan antara lain:

6. Menganalisis kinerja *real-time pipeline* Nvidia DeepStream pada deteksi kendaraan di Jetson Orin Nano.
7. Mengevaluasi pengaruh optimasi *pipeline*, khususnya penerapan NMS paralel berbasis TensorRT plugin (EfficientNMS), terhadap peningkatan performa pemrosesan *real-time*.

8. Mengevaluasi pengaruh pemilihan algoritma *tracking* terhadap efisiensi komputasi *real-time pipeline* dibandingkan konfigurasi *baseline*.

1.5 Manfaat

9. **Manfaat Teoritis:** Menjadi referensi ilmiah mengenai optimasi *pipeline* pemrosesan video *real-time* pada sistem *edge* untuk aplikasi ADAS, khususnya dalam penerapan Nvidia DeepStream, NMS paralel berbasis TensorRT plugin (EfficientNMS), serta evaluasi efisiensi komputasi pemilihan algoritma *tracking* pada perangkat Jetson Orin Nano.
10. **Manfaat Praktis:** Memberikan pedoman teknis bagi pengembang sistem *embedded* dan ADAS mengenai konfigurasi *pipeline* yang paling optimal untuk mencapai kinerja *real-time*, terutama dalam aspek latensi, *throughput*, dan stabilitas pemrosesan pada perangkat Jetson Orin Nano.
11. **Manfaat Institusional:** Berkontribusi dalam pengembangan riset terapan di bidang Visi Komputer, *Edge AI*, dan *Embedded Systems*, serta memperkuat kapasitas institusi dalam kajian pemanfaatan teknologi akselerasi perangkat keras NVIDIA pada sistem cerdas berbasis video.

1.6 Ruang Lingkup Penelitian

Agar penelitian lebih terarah dan fokus, maka batasan penelitian ini adalah sebagai berikut:

12. Pengujian kinerja perangkat keras difokuskan hanya pada satu perangkat *edge computing* berupa NVIDIA Jetson Orin Nano varian memori 4 gigabyte, dan tidak mencakup perangkat kelas lebih tinggi berikut modul akselerator perangkat keras khusus yang menyertainya.
13. Peningkatan kinerja dianalisis hanya melalui perbandingan *pipeline* sebelum dan sesudah dua bentuk optimasi, berupa penerapan mekanisme Non-Maximum

Suppression paralel berbasis plugin TensorRT dan penggantian algoritma pelacakan objek antara profil NvDCF dan profil NvSORT.

14. Penelitian menggunakan model deteksi yang telah dilatih sebelumnya sehingga tidak mencakup proses pelatihan model dari awal maupun peningkatan akurasi arsitektural model deteksi.
15. Penelitian hanya membahas pipeline perangkat lunak untuk pemrosesan video dan tidak mencakup integrasi dengan modul kendali kendaraan maupun subsistem bantuan pengemudi lain di luar komponen persepsi visual.
16. Evaluasi algoritma pelacakan objek dibatasi pada aspek efisiensi komputasi berupa laju bingkai per detik, latensi komponen pelacak, dan utilisasi sumber daya perangkat keras, dan tidak mencakup evaluasi kualitas maupun akurasi luaran pelacakan, karena penelitian ini tidak memiliki akses pada kumpulan data pelacakan objek berlabel dengan identitas objek yang konsisten antar-bingkai.
17. Seluruh pengujian dijalankan pada satu tingkat presisi numerik tunggal berupa setengah presisi (Half-Precision Floating-Point, FP16), dan tidak membandingkan konfigurasi presisi penuh maupun kuantisasi bilangan bulat 8-bit sebagai konfigurasi produksi alternatif.
18. Kamera stereo yang digunakan sebagai sumber video hanya dimanfaatkan sebagai sumber aliran video dua dimensi, sehingga kemampuan estimasi jarak berbasis stereo pada kamera tersebut tidak dimanfaatkan dan tidak menjadi bagian dari penelitian ini.

BAB II METODE PENELITIAN

2.1 Tempat dan Waktu Penelitian

Penelitian serta pengujian optimasi model inferensi dilaksanakan di Departemen Teknik Informatika, Universitas Hasanuddin. Keseluruhan tahapan penelitian, mulai dari persiapan lingkungan sistem, kompilasi *pipeline*, hingga pengambilan data performa perangkat eksekusi, dilakukan pada rentang waktu **Mei 2026 hingga Agustus 2026**.

nama laboratorium spesifik (mis. Laboratorium Jaringan dan Visi Komputer) belum disebutkan eksplisit baik di PDF maupun dokumen proyek ini. Hal ini perlu dikonfirmasi penulis bila diminta pembimbing/penguji.

Eksekusi eksperimen inti (pengujian *runtime/hardware* pada Jetson Orin Nano, 60 *run*) secara spesifik tercatat berlangsung pada **19 Agustus 2026**, pukul 11:33–12:46 WITA (± 72 menit, termasuk *cooldown* antar-*run*), sebagaimana tercatat pada metadata setiap *run* pengujian (lihat rincian kondisi eksekusi di Bab III). Tanggal ini berada di dalam rentang Mei–Agustus 2026 di atas, sebagai bagian akhir dari tahapan penelitian.

2.2 Instrumen Penelitian

Instrumen yang digunakan dalam penelitian ini adalah sebagai berikut:

19. **Perangkat lunak** a. Sistem Operasi Arch Linux 64-bit b. Sistem Operasi Windows 11 64-bit c. Sistem Operasi L4T (*Linux for Tegra* / Ubuntu Linux) d. *Notebook* Kaggle GPU NVIDIA Tesla P100 e. Zed Editor f. GitHub g. Zen Web Browser h. Microsoft Office 365 i. Canva j. Lucidchart k. Mendeley Reference Manager l. NVIDIA JetPack m. NVIDIA DeepStream SDK n. NVIDIA TensorRT o. NVIDIA Nsight Systems

20. **Perangkat keras** a. Laptop Advan WorkPlus, AMD Ryzen 5 6600H dengan Radeon Graphics (3,30 GHz), RAM 16GB, SSD 512GB b. NVIDIA Jetson Orin Nano Super 4GB, GPU Ampere, 512 CUDA core, 16 Tensor core (generasi ke-3), hingga 625 MHz
21. **Bahasa Pemrograman** a. C++ b. Python v3 c. Bash
22. **Library Backend** a. Ultralytics (YOLO) oleh Tim Ultralytics: digunakan untuk deteksi objek pada *frame* video. b. NumPy oleh Travis Oliphant: digunakan untuk komputasi numerik dan manipulasi *array*. c. Pillow: digunakan untuk menyimpan *snapshot frame* dan manipulasi gambar.

Rincian teknis perangkat Jetson Orin Nano. Identifikasi SKU dan mode daya perangkat butir 2b di atas didasarkan pada mode daya maksimum yang terbaca melalui utilitas pengaturan daya bawaan Jetson Linux (*nvpmodel*) pada perangkat, yaitu **10W**. Nilai ini unik untuk SKU **4GB** pada *firmware* standar (non-*Super clock mode*): modul 8GB pada *firmware* yang sama hanya memiliki mode 7W/15W (tidak ada mode 10W), sesuai tabel *Operating Requirements* pada *datasheet* resmi NVIDIA (NVIDIA, 2024). Karena mode maksimum yang terbaca adalah 10W (bukan 25W), perangkat dipastikan berjalan pada konfigurasi daya **default** (7W/10W), bukan mode performa maksimal *Super Clock* (JetPack 6.2+), sehingga performa GPU/CPU yang menjadi acuan Bab III mencerminkan batas atas mode *default* tersebut (hingga 625 MHz, sesuai butir 2b), bukan performa puncak SoC (hingga 1020 MHz). Identifikasi ini diturunkan dari pembacaan utilitas *nvpmodel*, bukan inspeksi label fisik modul. Penulis disarankan mengonfirmasi ulang melalui pembacaan berkas identitas perangkat pada *device tree* sistem saat perangkat kembali tersedia, sebagai verifikasi independen.

Tabel 2.1 Spesifikasi Jetson Orin Nano 4GB (NVIDIA, 2024)

Komponen	Mode <i>default</i> (7W/10W, dipakai penelitian ini)	Mode performa maksimal <i>Super Clock</i> (tidak dipakai)
GPU	Ampere, 512 CUDA core, 16	hingga 1020 MHz

	Tensor core (generasi ke-3), hingga 625 MHz	
Performa AI (INT8)	10 TOPS (<i>dense</i>) / 20 TOPS (<i>sparse</i>)	17 TOPS (<i>dense</i>) / 34 TOPS (<i>sparse</i>)
CPU	6-core Arm Cortex-A78AE (2 klaster: 4-core + 2-core), <i>system cache</i> 4MB, hingga 1,5 GHz/core	hingga 1,7 GHz/core
Memori	4GB LPDDR5, bus 64-bit, <i>bandwidth</i> hingga 34 GB/s	hingga 51 GB/s
Mode daya tersedia	7W / 10W / 25W (<i>Super Clock</i>)	tidak tersedia
DLA	Tidak tersedia	Tidak tersedia

Sumber: NVIDIA (2024), tabel "AI Performance", "GPU Operation" (Tabel 2-1), "CPU Cluster", "Memory Subsystem", dan "Operating Requirements". Berbeda dari varian Jetson Orin NX dan Jetson AGX Orin, Jetson Orin Nano **tidak dilengkapi Deep Learning Accelerator (DLA)**, sehingga strategi optimasi yang bergantung pada DLA (seperti pada proposal awal penelitian ini yang menargetkan Jetson AGX Orin) tidak dapat diterapkan pada perangkat ini (lihat pembahasan latar belakang dan keterbatasan penelitian pada Bab I).

Versi library inti pipeline (pelengkap butir 1l–1n). Inference SDK: NVIDIA DeepStream 7.1 (berbasis GStreamer 1.0, GLib 2.0); precision runtime: FP16 (TensorRT); bahasa aplikasi utama: C++17, dibangun menggunakan sistem build CMake versi 3.10 ke atas; parser custom untuk keluaran EfficientNMS memerlukan CUDA Toolkit yang dicari dan ditautkan secara otomatis pada proses kompilasi; evaluasi akurasi proxy FP32 memakai fungsi validasi bawaan kerangka kerja

Ultralytics; evaluasi akurasi as-deployed FP16 memakai pustaka evaluasi standar COCO; profiling hardware memakai tegrastats bawaan Jetson Linux. Versi persis GStreamer/GLib/CUDA Toolkit yang terpasang di perangkat pengujian tidak dipin secara eksplisit pada konfigurasi build proyek (memakai versi sistem yang terdeteksi otomatis saat kompilasi). Penulis perlu mencatat versi aktual komponen-komponen tersebut sebagai bagian dari metadata reproducibility.*

Justifikasi pemilihan presisi FP16. TensorRT mengoptimalkan model terlatih (melalui *layer fusion*, *kernel auto-tuning*, dan kuantisasi presisi) menjadi *engine* yang dieksekusi efisien pada GPU target. Tiga tingkat presisi umum yang didukung adalah FP32 (presisi penuh, *baseline* akurasi), FP16 (setengah presisi, memanfaatkan *Tensor Core* untuk percepatan dengan penurunan akurasi yang umumnya dapat diabaikan), dan INT8 (kuantisasi bilangan bulat 8-bit, tercepat namun membutuhkan dataset kalibrasi tambahan dan berisiko kehilangan akurasi lebih besar). Penelitian ini menggunakan presisi **FP16** secara konsisten di seluruh model uji, dengan pertimbangan: Jetson Orin Nano tidak memiliki DLA sehingga keuntungan INT8 khas DLA tidak berlaku, sementara FP32 tidak memanfaatkan penuh *Tensor Core* GPU Ampere pada perangkat ini untuk keuntungan akurasi yang hampir tidak terasa. Perbandingan akurasi FP32-*proxy* vs. FP16 *as-deployed* menjadi bagian dari skenario pengujian akurasi.

2.3 Tahapan Penelitian

Penelitian ini merupakan penelitian **eksperimental kuantitatif** yang membandingkan kinerja konfigurasi *pipeline* deteksi-dan-tracking objek secara berpasangan (*baseline* vs. *optimized*) pada instrumen dan kondisi pengujian yang dikontrol seketat mungkin: perangkat keras yang sama (Jetson Orin Nano), video input yang sama, mode daya yang sama (mode daya terkunci dan frekuensi *clock* dikunci pada nilai maksimal melalui utilitas *nvpmodel* dan *jetson_clocks*), dan *harness* pengukuran yang sama untuk seluruh skenario. Desain ini dipilih karena

ketiga rumusan masalah pada dasarnya menanyakan **efek dari mengganti satu komponen pipeline** terhadap kinerja real-time**.

Tahapan penelitian disusun sebagai alur kerja terstruktur dari tahap studi literatur hingga pelaporan, terdiri atas delapan langkah:

23. **Studi Literatur:** mengkaji penelitian terdahulu terkait sistem bantuan pengemudi (ADAS), optimasi *pipeline* pemrosesan video *real-time* pada sistem *edge*, implementasi model keluarga YOLO untuk deteksi objek, serta metode akselerasi algoritma *Non-Maximum Suppression* (NMS) dan algoritma *tracking*.
24. **Identifikasi Kebutuhan Penelitian:** merumuskan kebutuhan penelitian secara sistematis untuk menetapkan permasalahan, tujuan, spesifikasi perangkat keras (Jetson Orin Nano 4GB), serta ruang lingkup penelitian.
25. **Pengumpulan Data:** persiapan data yang akan dievaluasi, mencakup penyiapan dataset KITTI dengan pemetaan ulang tiga kelas kustom (mobil, van, truk) untuk evaluasi akurasi, serta penyiapan berkas video pengujian untuk proses *benchmark* performa *runtime*.
26. **Perancangan Sistem:** merancang arsitektur *pipeline* sistem DeepStream secara menyeluruh, meliputi skema input video dan inferensi menggunakan model YOLO (v8n, v9t, v10n, v26n).
27. **Optimasi Pipeline:** mengimplementasikan dan mengoptimalkan rancangan *pipeline* menggunakan bahasa C++ dan kerangka kerja GStreamer, meliputi penerapan *plugin* EfficientNMS_TRT untuk mengeksekusi proses NMS secara paralel pada GPU dan konfigurasi algoritma *tracker* untuk membandingkan efisiensi komputasi.
28. **Pengujian Sistem:** mengevaluasi kinerja *pipeline* pada lingkungan Jetson Orin Nano melalui tiga skenario utama: pengukuran *baseline* model deteksi, pengukuran performa setelah penerapan NMS paralel, dan evaluasi efisiensi komputasi antara algoritma *tracking* NvDCF dan NvSORT.

29. **Analisis dan Interpretasi Hasil:** menginterpretasikan luaran evaluasi secara komprehensif melalui ekstraksi log utilisasi perangkat keras (*tegrastats*) dan log metrik *runtime* per-bingkai. Data dianalisis untuk membandingkan *throughput* (FPS), latensi *end-to-end* maupun per-komponen, utilisasi perangkat keras (CPU, GPU, RAM, daya), serta menganalisis *trade-off* antara kecepatan dan akurasi.
30. **Penyusunan Laporan:** mendokumentasikan seluruh proses dan hasil penelitian secara sistematis dan terstruktur ke dalam bentuk laporan akhir.

Sesuai rekomendasi protokol *benchmark* yang telah ditetapkan sejak tahap perancangan pengujian, setiap skenario pada langkah 6 diulang **5 repetisi** dengan jeda *cooldown* 60 detik antar repetisi/skenario (stabilisasi termal), dan 10 detik awal tiap *run* (*warm-up*) dibuang saat agregasi (lihat kondisi eksekusi aktual di Bab III).

2.4 Teknik Pengambilan Dataset

Pengumpulan data dalam penelitian ini sepenuhnya menggunakan data sekunder yang bersumber dari dataset publik **KITTI Vision Benchmark Suite**. Pemilihan sumber data ini didasarkan pada karakteristik penelitian yang menggunakan model *pre-trained* dan berfokus pada optimasi performa *pipeline deployment* di perangkat *edge*, sehingga pengumpulan dataset diarahkan untuk memenuhi kebutuhan pengujian akurasi deteksi sebagai *baseline proxy*, bukan untuk keperluan pelatihan model dari awal. Dalam proses implementasinya, dilakukan pemetaan ulang terhadap anotasi asli KITTI menjadi tiga kelas kustom yang disesuaikan dengan tujuan penelitian, yaitu kelas *car* (label 0), kelas *van* (label 1), dan kelas *truck* (label 2), yang didefinisikan pada berkas definisi label kustom proyek. Proses *re-mapping* ini penting dilakukan mengingat anotasi bawaan KITTI memiliki kategori objek yang lebih beragam sehingga perlu disederhanakan agar selaras dengan skema klasifikasi yang digunakan dalam sistem deteksi yang dikembangkan.

Distribusi pembagian berkas citra yang digunakan dalam penelitian ini terdiri atas **5.788 citra untuk data latih** dan **1.010 citra untuk data validasi**. Pada *subset* data validasi yang berjumlah 1.010 citra tersebut, teridentifikasi total **4.722 instance** objek dengan rincian distribusi kelas yang menunjukkan ketimpangan (*class imbalance*) yang cukup signifikan: kelas *car* mendominasi dengan 4.167 *instance*, diikuti kelas *van* sebanyak 405 *instance*, dan kelas *truck* sebanyak 150 *instance*. *Split* validasi ini identik untuk seluruh model, sehingga hasil akurasi antar model dapat dibandingkan langsung. Ketimpangan distribusi kelas ini menjadi salah satu aspek penting yang perlu diperhatikan dalam interpretasi hasil evaluasi model, karena performa deteksi pada kelas dengan jumlah *instance* lebih sedikit berpotensi menghasilkan metrik yang kurang representatif dibandingkan kelas yang mendominasi dataset, sebagaimana dibahas lebih lanjut pada bagian keterbatasan penelitian Bab I.

2.5 Perancangan dan Implementasi Sistem

2.5.1 Konfigurasi Edge Device

Perangkat keras utama yang digunakan sebagai *edge device* dalam penelitian ini adalah NVIDIA Jetson Orin Nano varian 4GB. Perangkat ini dioperasikan pada mode daya standar 7W/10W, yang dikonfigurasi melalui indeks mode daya pertama pada utilitas *nvpmode*. Frekuensi *clock* perangkat juga dikunci pada tingkat maksimal menggunakan utilitas *jetson_clocks* untuk memastikan stabilitas performa selama pengujian berlangsung. Sebagai sumber masukan video, sistem ini menggunakan kamera Stereolabs ZED (stereo, dipakai hanya sebagai sumber *stream* 2D; lihat batasan pada Bab I) yang diatur untuk menghasilkan aliran video pada kecepatan 30 *Frames Per Second* (FPS); kamera ini juga mendukung kecepatan bingkai 15/30/60/100/120 FPS yang dapat dikonfigurasi, dengan 30 FPS ditetapkan sebagai kecepatan *default* konfigurasi *benchmark*.

Lingkungan perangkat lunak yang diimplementasikan pada sistem ini dibangun di atas kerangka kerja GStreamer 1.0 dan memanfaatkan NVIDIA DeepStream SDK versi 7.1. Untuk menunjang eksekusi model *deep learning* secara efisien, TensorRT digunakan dengan pengaturan tingkat presisi *Half-Precision Floating-Point* (FP16); lihat justifikasi lengkap pada subbab Instrumen Penelitian. Keseluruhan program dan perancangan *pipeline* diimplementasikan menggunakan bahasa pemrograman C++17.

2.5.2 Persiapan Model dan Dataset

Persiapan data pada sistem ini memanfaatkan dataset KITTI yang telah dipetakan ulang menjadi tiga kelas objek khusus kendaraan. Empat varian model arsitektur YOLO kelas *nano* dan *tiny* dievaluasi dalam penelitian ini, dipilih karena kelas ukuran model yang setara (anggaran komputasi kira-kira sebanding), sehingga selisih luaran antar model lebih mencerminkan perbedaan arsitektur generasi ketimbang perbedaan skala model:

Model	Params	GFLOPs	Konfigurasi NMS
YOLOv8n	3.006.233	8,1	NMS bawaan <i>nvinfer</i> (<i>baseline</i>)
YOLOv9t	1.971.369	7,6	NMS bawaan <i>nvinfer</i> (<i>baseline</i>)
YOLOv10n	2.265.753	6,5	NMS-free (arsitektural)
YOLO26n	2.375.421	5,2	NMS-free (arsitektural)
YOLOv8n + EfficientNMS	3.006.233	8,1	NMS paralel (<i>EfficientNMS_TRT</i>)

YOLOv9t + EfficientNMS	1.971.369	7,6	NMS paralel (<i>EfficientNMS_TRT</i>)
---------------------------	-----------	-----	--

Total 6 konfigurasi model di atas menjadi dasar 12 skenario RM3 (6 model $\times$ 2 *tracker*). Perbandingan RM2 (NMS standar vs. paralel) hanya berlaku pada pasangan YOLOv8n/YOLOv9t (model dengan varian *baseline* dan EfficientNMS yang setara). YOLOv10n dan YOLO26n tidak disertakan pada perbandingan RM2 karena keduanya sudah NMS-*free* secara arsitektural, sehingga tidak ada pasangan pembanding yang setara.

Pada saat *pipeline* dijalankan untuk pertama kalinya, proses konversi model dilakukan secara otomatis: format model awal ONNX dikompilasi menjadi *engine* TensorRT berpresisi FP16 yang spesifik untuk arsitektur GPU target (spesifik perangkat/versi TensorRT, dihapus ulang bila berpindah perangkat). Perlu dicatat secara khusus bahwa model YOLOv10n dan YOLO26n secara arsitektural telah didesain dengan sifat NMS-*free*: kedua model tersebut tidak lagi membutuhkan tahapan NMS terpisah pada *post-processing*, karena arsitektur jaringan telah dilatih secara inheren (*consistent dual assignments*) untuk menghasilkan prediksi akhir tanpa duplikasi *bounding box*. Poin ini relevan untuk interpretasi luaran RM2 di Bab III, karena optimasi NMS (EfficientNMS) secara inheren tidak berlaku sama untuk model NMS-*free*.

Satu model tambahan, **YOLOv8n-COCO** (80 kelas), dipertahankan hanya sebagai *sanity-check* umum kebenaran *pipeline* (mis. memverifikasi *parser/rendering* berjalan benar di luar domain KITTI), **bukan** kompetitor pada perbandingan akurasi/efisiensi utama.

2.5.3 Implementasi Pipeline Baseline

Arsitektur aliran data atau *pipeline baseline* yang dirancang dibangun berdasarkan elemen-elemen standar GStreamer dalam kerangka DeepStream. Aliran

proses direpresentasikan mulai dari elemen masukan video (kamera ZED atau berkas rekaman), yang kemudian diteruskan ke elemen *multiplexer* aliran (*nvstreammux*) untuk tahap *batching*. Aliran data selanjutnya diproses oleh elemen inferensi utama (*nvinfer*) sebagai *Primary GPU Inference Engine* (GIE) untuk melakukan inferensi deteksi, dilanjutkan ke elemen pelacak objek (*nvtracker*, profil *default* *NvDCF*) untuk pelacakan objek, elemen konversi format (*nvvideoconvert*), dan elemen penampil (*nvsosd*) untuk penggambaran *bounding box* serta label pada *frame*. Aliran diakhiri pada elemen *output sink* (RTSP, layar lokal, atau berkas MP4) untuk menampilkan atau menyimpan hasil.

Penekanan utama pada implementasi arsitektur ini adalah pemanfaatan *buffer NVMM (NVIDIA Memory Manager)* yang memfasilitasi operasi *zero-copy*. Melalui integrasi *buffer* tersebut, keseluruhan tahapan pemrosesan mulai dari *decode* hingga *tracking* dieksekusi dan dijaga sepenuhnya di dalam memori GPU, sehingga perpindahan data secara bolak-balik antara CPU dan GPU yang memakan waktu komputasi besar dapat dihindari, sehingga lebih efisien untuk kebutuhan *real-time* pada SoC *embedded* dibanding *pipeline* OpenCV manual yang biasanya bolak-balik CPU ↔ GPU tiap tahap. Karakteristik ini menjadi alasan utama pemilihan DeepStream dibanding *pipeline* manual berbasis OpenCV dan TensorRT/ONNXRuntime.

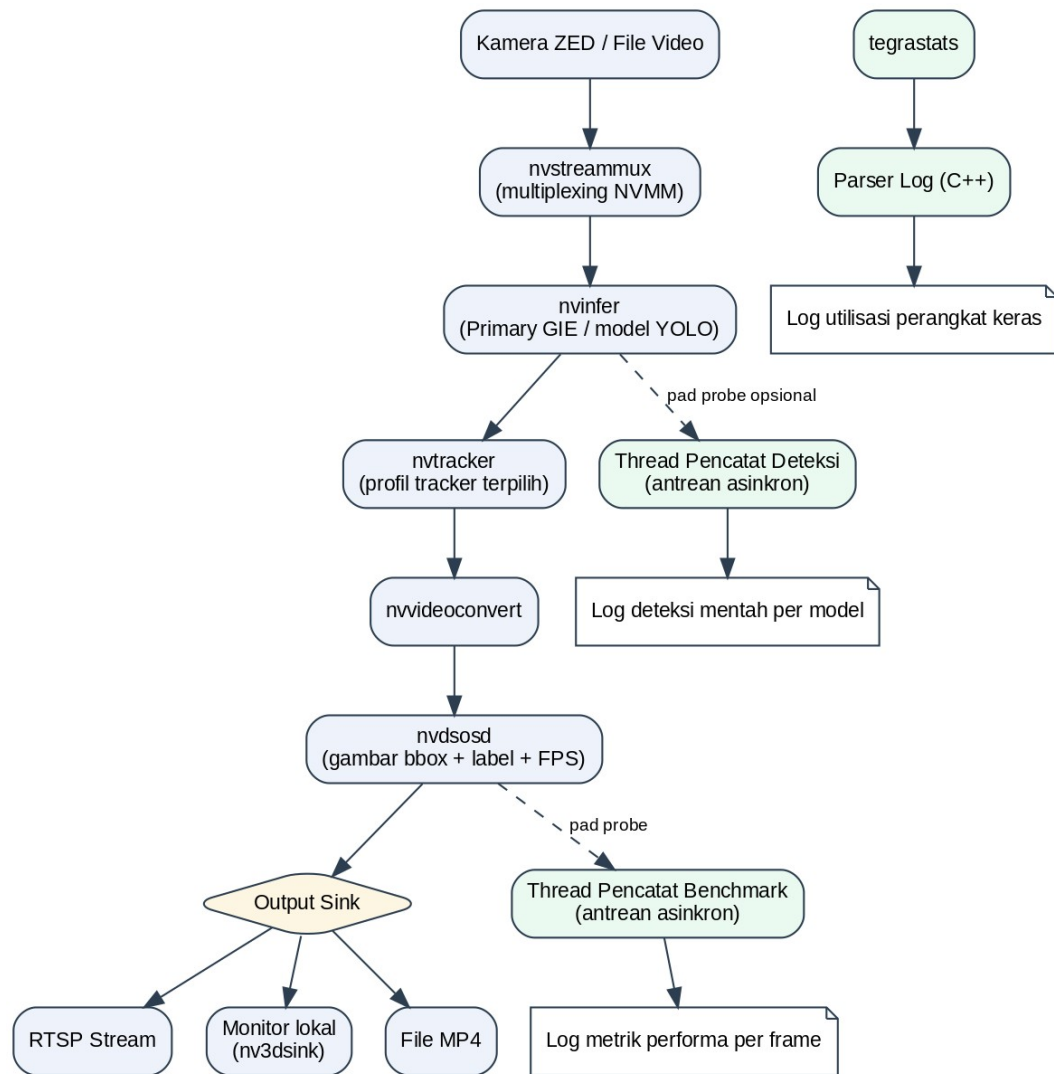
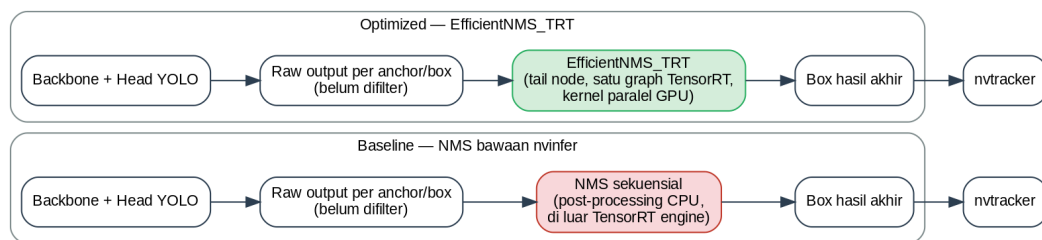


Diagram alur pipeline DeepStream inti beserta thread pencatatan benchmark/deteksi dan parser tegrastats.

Implementasi inti *pipeline* diwujudkan dalam sebuah kelas aplikasi utama berbahasa C++ yang menyusun tahapan pembentukan *pipeline* secara modular, meliputi konstruksi elemen masukan, elemen pemrosesan inti, dan elemen keluaran secara terpisah.

2.5.4 Implementasi Optimasi Efficient NMS

Untuk meminimalisasi latensi pada tahap *post-processing*, implementasi optimasi diterapkan menggunakan *plugin* bawaan TensorRT, yakni **EfficientNMS_TRT**. Penerapan *plugin* ini secara spesifik dikhususkan pada arsitektur YOLOv8n dan YOLOv9t yang masih mensyaratkan tahap penyaringan *bounding box* secara manual. Cara kerja optimasi ini adalah memindahkan eksekusi algoritma *Non-Maximum Suppression* (NMS), yang biasanya beroperasi secara sekuensial dan membebani komputasi CPU, menjadi operasi paralel penuh di dalam *kernel* GPU sebagai bagian turunan (*tail node*) dari *graph* TensorRT. Motivasi ini mendasari rumusan masalah #2 pada Bab I. Dengan pendekatan ini, latensi *post-processing* dapat direduksi karena tahap NMS diintegrasikan langsung ke dalam *engine* inferensi, tanpa mengubah arsitektur ONNX/*engine* baseline (EfficientNMS_TRT dipasang sebagai *tail* tambahan yang bergantung pada *output* detektor).



Perbandingan posisi tahap NMS pada varian baseline versus EfficientNMS_TRT di dalam pipeline.

Diagram di atas menegaskan poin utama teks: EfficientNMS_TRT bukan komponen *pipeline* GStreamer tambahan, melainkan perubahan **di dalam batas nvinfer**: NMS berpindah dari tahap terpisah setelah engine (kotak merah, CPU, sekuensial) menjadi tail node yang menyatu di dalam satu graph TensorRT yang sama dengan backbone/head (kotak hijau, GPU, paralel). Elemen pipeline di hilir (nvtracker* dst.) tidak berubah pada kedua varian.

Diagram di atas menunjukkan **posisi** perubahan pada *pipeline*, namun belum menunjukkan **bagaimana** fusi tersebut sebenarnya terjadi. Perlu ditekankan bahwa fusi ini adalah proses **build-time** (satu kali, di luar *runtime pipeline*), dilakukan oleh sebuah utilitas Python terpisah yang memanfaatkan TensorRT Python API, **bukan** proses yang berjalan ulang tiap *frame*:

Build-time — sekali, offline (utilitas pembangun engine)

Model ONNX
backbone + head, TIDAK diubah

1. parse graph ONNX

2. Temukan tensor internal pre-NMS
[1, 8400, 7] = x1,y1,x2,y2,
score_class0..2
(sebelum output baseline [1,8400,6])

3. tambahkan node plugin

4. Pasang node EfficientNMS_TRT
pada definisi jaringan TensorRT,
disambung langsung ke tensor internal itu

5. bangun jaringan terserialisasi

Engine EfficientNMS
engine BARU & terpisah — ONNX
dan engine baseline tetap utuh

di-load nvinfer sbg satu engine

Runtime — per frame, di dalam nvinfer

Backbone + Head YOLO

Alur fusi EfficientNMS_TRT pada tahap build-time (offline) dan eksekusinya pada tahap runtime (per frame).

Tiga hal yang ditegaskan diagram ini: (1) fusi dilakukan **sekali di build-time** terhadap *network definition* TensorRT, bukan operasi *runtime*, sehingga biaya penggabungan graph itu sendiri **tidak** menyumbang latensi *pipeline* saat pengujian berjalan; (2) *node plugin* disambungkan ke **tensor internal** graph (bukan ke output akhir baseline yang sudah difilter, berdimensi [B,N,6]), sehingga tensor pre-NMS berdimensi [1,8400,7] tidak pernah keluar dari *engine*/tidak perlu transfer host↔device tambahan sebelum NMS dijalankan; (3) berkas ONNX sumber dan *engine baseline* **tidak disentuh**. Luaran fusi disimpan sebagai berkas *engine* baru dan terpisah, sehingga kedua varian (*baseline* dan EfficientNMS) tetap dapat dijalankan berdampingan untuk perbandingan (skenario 2 pada pengujian).

Pemanggilan API TensorRT yang digambarkan pada diagram, mulai dari penambahan *node plugin* ke definisi jaringan hingga proses pembangunan jaringan terserialisasi, telah diverifikasi kesesuaiannya dengan implementasi aktual pada utilitas pembangun *engine* yang dikembangkan.

Pendekatan ini **berbeda** dari yang digambarkan pada diagram arsitektur proposal awal (kernel CUDA kustom dengan tahap *ParallelDispatch* → *Workers evaluasi pasangan IoU* → *Custom Map Kernel* → *ParallelReduce*). Proposal menggambarkan kernel paralel yang ditulis dari nol, sedangkan implementasi final memakai *plugin* siap pakai vendor. Perbedaan ini perlu dijelaskan secara konsisten pada Bab I dan III.

Implementasi konkret melibatkan dua bagian: (1) *parser custom* yang membaca *output* tambahan dari *tail node* EfficientNMS_TRT pada *engine* TensorRT, dan (2) berkas konfigurasi *nvinfer* khusus untuk kedua model yang mendukung varian ini. Model YOLOv10n dan YOLO26n tidak memerlukan integrasi ini karena keduanya sudah *NMS-free* secara arsitektural.

2.5.5 Implementasi Multi Object Tracking

Tahap *object tracking* diimplementasikan dengan mengintegrasikan elemen *nvtracker* yang diletakkan tepat setelah proses deteksi oleh *nvinfer*. Implementasi ini difokuskan pada pengonfigurasi dan perbandingan efisiensi dua algoritma pelacakan utama dari DeepStream:

Tracker	Karakteristik singkat
NvDCF	<i>Feature-based</i> , pemrosesan piksel penuh, akurasi/ <i>robustness</i> tertinggi
NvSORT	<i>Motion-only</i> (Kalman filter + algoritma Hungarian), tanpa pemrosesan piksel, paling ringan

Algoritma **NvDCF** menggunakan pendekatan berbasis fitur (*feature-based*) yang melibatkan pemrosesan data piksel secara penuh untuk mengekstraksi korelasi visual objek, sehingga memiliki tingkat akurasi yang tinggi namun mensyaratkan komputasi yang berat; profil ini menjadi *baseline* implementasi proyek. Di sisi lain, **NvSORT** dievaluasi menggunakan pendekatan murni berbasis pergerakan (*motion-only*) yang diturunkan dari Kalman filter dan algoritma asosiasi Hungarian, sehingga dapat dieksekusi dengan sangat ringan secara komputasi karena beroperasi tanpa pemrosesan piksel sama sekali. Kedua profil ini dipilih karena mewakili dua ujung spektrum efisiensi komputasi *tracker* yang tersedia di elemen *nvtracker* DeepStream, dan konfigurasi masing-masing profil sudah tersedia di repositori proyek sejak awal implementasi.

Ditegaskan bahwa implementasi pelacakan ini difokuskan **murni untuk mengukur metrik efisiensi komputasi**, yaitu penggunaan sumber daya (*resource*) dan latensi pemrosesan pada perangkat *edge*, sesuai batasan penelitian pada Bab I. Sistem evaluasi pelacakan ini **tidak** dirancang untuk mengukur metrik kualitas

ketepatan pelacakan itu sendiri, seperti jumlah *ID Switch* maupun *Multiple Object Tracking Accuracy* (MOTA)/IDF1.

Pembatasan ini bukan sekadar konsekuensi keterbatasan dataset, melainkan konsisten dengan definisi sumbu perbandingan yang sudah melekat pada desain kedua profil *tracker* itu sendiri. Dokumentasi resmi NVIDIA (*Gst-nvtracker* plugin manual, DeepStream SDK) menyatakan NvSORT "tidak melibatkan pemrosesan data piksel sama sekali" sehingga "efisien secara komputasi", sedangkan NvDCF memakai *visual tracker* berbasis *discriminative correlation filter*, yaitu pemrosesan fitur piksel per objek per *frame* (NVIDIA, *DeepStream SDK Plugin Manual*, bagian *Gst-nvtracker*). Postingan blog resmi NVIDIA Developer soal DeepStream SDK 6.2 juga secara eksplisit menempatkan NvSORT sebagai "*lightweight, CPU-only implementation but still competitively accurate*" dan NvDCF sebagai penghasil "*best accuracy and robustness*" lewat kombinasi *conventional ML* (DCF) dan *deep learning* (ReID) (Shin & Li, 2023). Hal ini mengonfirmasi bahwa sumbu akurasi-vs-komputasi ini memang didesain vendor sebagai *trade-off* yang sudah diketahui karakteristiknya secara kualitatif, bukan sesuatu yang perlu diukur ulang oleh penelitian ini untuk membuat pertanyaan "*tracker* mana yang lebih efisien secara komputasi" valid dijawab.

Pendekatan mengevaluasi pilihan algoritma murni dari sisi biaya komputasi, dengan karakter akurasi/kualitas sudah diketahui dan tidak diukur ulang, juga punya preseden metodologis yang mapan di luar domain *tracking*: **MLPerf Mobile Inference Benchmark** (Janapa Reddi dkk., 2022) menetapkan pada setiap tugas pengujian sebuah ambang akurasi dan kualitas minimum sebagai syarat kelulusan tetap, sementara metrik yang secara eksplisit dibandingkan dan dilaporkan meningkat antar generasi submisi adalah latensi dan *throughput*. Struktur ini serupa dengan penelitian ini, karena karakteristik akurasi setiap *tracker* sudah didokumentasikan kualitatif oleh vendor, dan kontribusi penelitian ini adalah mengkuantifikasi biaya komputasinya secara spesifik pada perangkat Jetson Orin

Nano yang belum ada di literatur. Preseden ini relevan pula karena MLPerf Mobile secara khusus menyasar evaluasi performa pada perangkat *on-device/edge* dengan sumber daya terbatas, konteks yang sejalan dengan platform Jetson Orin Nano pada penelitian ini, alih-alih server kelas *datacenter*.

Adapun ketidaktersediaan dataset ber-anotasi *track ID* berurutan tetap menjadi alasan pendukung (bukan alasan tunggal): metrik kualitas *tracking* (MOTA/IDF1/*ID switch*) secara struktural membutuhkan *ground truth* dengan *field* identitas objek yang konsisten antar *frame* (mis. *benchmark* KITTI Tracking, MOTChallenge MOT16/17/20), berbeda dari dataset deteksi *single-frame* seperti KITTI 2D Object Detection yang dipakai penelitian ini (*bounding box* per *frame* tanpa *field ID* lintas-*frame*), sehingga menambah metrik ini bukan sekadar menghitung ulang, melainkan membutuhkan dataset dan proses anotasi baru yang di luar ruang lingkup penelitian.

2.6 Pengujian

Sistem evaluasi dan *benchmarking* dirancang melalui pengembangan aplikasi kustom berbasis bahasa C++ yang mengadopsi mekanisme *pad probe* dari antarmuka pemrograman aplikasi (*Application Programming Interface*, API) GStreamer, sebuah titik intersepsi yang memungkinkan pembacaan data pada aliran pemrosesan tanpa mengubah alur eksekusi *pipeline* itu sendiri. Agar tidak menyebabkan blokir (*blocking*) atau mengganggu jalur *rendering* kritis pada *pipeline* utama, pencatatan metrik per bingkai (*frame*) dilaksanakan secara asinkron pada *thread* pencatatan terpisah yang memakai struktur data antrean: *pad probe* hanya menyalin data mentah ke antrean (operasi yang sangat cepat) dan langsung mengembalikan kendali ke *pipeline*, sementara *thread* terpisah mengambil data dari antrean dan menuliskannya secara asinkron ke media penyimpanan. Prinsip yang sama diterapkan baik pada pencatatan metrik performa maupun pada pencatatan hasil deteksi mentah untuk keperluan verifikasi akurasi (lihat skenario 4).

Instrumentasi dan otomasi pengujian. Pelaksanaan pengujian didukung oleh serangkaian utilitas otomasi yang dikembangkan penulis. Mekanisme uji *one-run* menjalankan *pipeline* dalam mode *benchmark*, merekam log performa per-*frame* dan log utilisasi perangkat keras secara konkuren, serta mencatat metadata setiap proses pengujian (model, konfigurasi, jenis *tracker*, mode input/keluaran, durasi eksekusi, versi kode sumber, dan mode daya perangkat). Setiap proses pengujian disimpan pada lokasi tersendiri yang tidak pernah menimpa hasil pengujian sebelumnya, guna menjaga keterlacakan (*traceability*) dan reproduksibilitas data. Di atas mekanisme ini, dibangun pula lapisan orkestrasi yang menjalankan seluruh dua belas skenario pengujian (enam model $\times$ dua konfigurasi *tracker*) secara berurutan dan otomatis, disertai penguncian mode daya perangkat di awal serta jeda pendinginan (*cooldown*) dan pembersihan tembolok (*cache*) sistem di antara setiap skenario untuk menjaga stabilitas termal selama pengujian berlangsung. Sebagai pelengkap, dikembangkan pula utilitas pendukung untuk verifikasi akurasi *as-deployed* FP16 (lihat status implementasi pada skenario 4).

Skenario pengujian:

31. *Baseline pipeline (RM1): empat model (YOLOv8n, YOLOv9t, YOLOv10n, YOLO26n), tracker default NvDCF, video input tetap; diukur throughput keseluruhan dan latensi pada tiap tahap komponen perangkat lunak pipeline*.*
32. **NMS standar vs. NMS paralel EfficientNMS (RM2):** pasangan YOLOv8n/YOLOv9t *baseline* vs. varian EfficientNMS, kondisi lain identik terhadap skenario 1. Fokus perbandingan pada latensi tahap inferensi (tahap yang mencakup *post-processing* NMS) dan *throughput* keseluruhan.
33. *Efisiensi komputasi tracking (RM3): seluruh dua belas skenario (enam model $\times$ dua konfigurasi tracker: NvDCF/NvSORT) dijalankan secara otomatis. Dibandingkan latensi tahap tracking serta utilisasi perangkat keras (persentase GPU, CPU per-core, RAM, dan daya) antar konfigurasi tracker. Tidak mengukur*

kualitas/akurasi tracking (ID switch, MOTA/IDF1); lihat batasan penelitian pada Bab I dan pembahasan pemilihan tracker sebelumnya.*

34. *Verifikasi akurasi as-deployed FP16 (pendukung, bukan bagian dari rumusan masalah inti): mengukur mAP langsung dari keluaran metadata deteksi pipeline DeepStream FP16 pada 1.010 gambar val KITTI yang sama, dibandingkan terhadap proxy FP32, untuk memastikan tidak ada penurunan akurasi tersembunyi akibat kuantisasi maupun proses penguraian (parsing*) metadata kustom.*

Status implementasi (per 2026-08-14): infrastrukturnya **sudah selesai diimplementasikan**, mencakup mekanisme pencatatan hasil deteksi mentah (kelas, kotak pembatas/*bounding box*, tingkat keyakinan/*confidence*) per *frame* melalui *thread* asinkron terpisah; utilitas konversi kumpulan gambar validasi menjadi satu berkas video tanpa kompresi (*lossless*) beserta pemetaan indeks bingkai ke gambar asli, sehingga dapat diputar lewat jalur input video yang sudah tersedia tanpa perlu membangun dukungan pemrosesan rangkaian-gambar (*image-sequence*) baru pada *pipeline C++*; serta utilitas yang mengonversi hasil pencatatan deteksi menjadi format evaluasi standar COCO, melakukan penskalaan-ulang (*rescale*) *bounding box* dari kanvas pemrosesan kembali ke resolusi gambar asli per *frame*, lalu menghitung mAP secara keseluruhan maupun per kelas.

Yang belum dilakukan (per 2026-08-21, lihat status terkini pada bagian relevan di Bab III): (a) ekspor 1.010 gambar val beserta anotasinya dari lingkungan pelatihan ke perangkat lokal/Jetson, perlu memastikan pembagian data (*split*) yang diekspor **identik** dengan pembagian data yang menghasilkan angka Bab III; (b) eksekusi nyata di Jetson untuk keempat model; (c) verifikasi visual (*sanity-check*) hasil penskalaan-ulang *bounding box* sebelum angka mAP dipercaya; (d) pemutakhiran dokumentasi hasil akurasi dan batasan penelitian dengan hasil aktual. Karena statusnya infrastruktur-siap-tapi-belum-dieksekusi, skenario ini ditulis di sini sebagai **rencana pengujian pendukung**, bukan hasil yang sudah ada.

2.7 Kriteria Evaluasi

Definisi konseptual metrik (*throughput*/FPS, latensi dan P95, *precision*, *recall*, mAP) telah dibahas pada Bab I, pada subbab Metrik Evaluasi Kinerja Real-Time Pipeline. Subbab ini **tidak mendefinisikan ulang** istilah tersebut, melainkan merumuskan bagaimana metrik itu diterapkan dan dinilai selama pelaksanaan pengujian pada subbab Pengujian, beserta kriteria yang dipakai untuk menilai luarannya terhadap masing-masing rumusan masalah. Kriteria evaluasi dikelompokkan menjadi dua kategori, yaitu **performa runtime dan perangkat keras (fokus utama RM1–RM3) serta kualitas deteksi** (kriteria pendukung pada skenario 4).

Metrik performa runtime dan perangkat keras. Metrik utama yang diukur adalah throughput dalam satuan bingkai per detik (Frames Per Second, FPS), dengan target laju waktu nyata ≥ 30 FPS, mengikuti konfigurasi default kamera ZED dan sejalan dengan tuntutan lapisan persepsi (perception layer) pada sistem safety-critical seperti ADAS, yang menuntut margin waktu tanggap yang konsisten demi mendukung keselamatan berkendara (Costa dkk., 2025). Di samping throughput, diukur pula latensi end-to-end serta latensi pada tiap tahap komponen perangkat lunak pipeline secara terpisah, meliputi tahap pra-multiplexing, multiplexing aliran video, inferensi model, tracking objek, pra-OSD, penyusunan OSD (on-screen display), dan output, yang diperoleh melalui pad probe GStreamer pada elemen-elemen kunci pipeline. Nilai rerata (mean) rentan menyembunyikan variabilitas ekstrem (jitter) yang krusial pada sistem safety-critical, sehingga latensi turut dilaporkan sebagai persentil ke-95 (P95), sejalan dengan praktik penetapan sasaran tingkat layanan (service level objective, SLO) latensi pada layanan inferensi di lingkungan edge (Nigade dkk., 2024), yang mengindikasikan bahwa 95% dari seluruh siklus frame diselesaikan dalam durasi kurang dari atau sama dengan nilai tersebut.*

Pengukuran perangkat keras dilakukan secara konkuren untuk merekam persentase penggunaan unit GPU, penggunaan CPU per-core, pemakaian kapasitas RAM, serta tingkat konsumsi daya per-jalur (*rail*), menggunakan utilitas pemantauan bawaan Jetson Linux dengan interval pencuplikan 1.000 milidetik, sejalan dengan preseden metodologis evaluasi konsumsi daya pada perangkat *embedded* untuk beban kerja visi komputer *real-time* (Suder dkk., 2023). Utilitas pemantauan bawaan ini dipilih dibanding utilitas pemantauan GPU generik karena yang disebut terakhir tidak tersedia pada arsitektur *driver* Jetson, sedangkan utilitas bawaan Jetson merupakan perangkat resmi dengan *overhead* rendah yang mampu membaca jalur daya *on-SoC* secara langsung tanpa instrumen tambahan.

Metrik kualitas deteksi. Sebagai kriteria pendukung pada skenario 4, kualitas deteksi diukur melalui *precision*, *recall*, dan *mean Average Precision* (mAP) pada *Intersection over Union* (IoU) *threshold* 0,5 serta rentang 0,5:0,95, dihitung dengan pustaka evaluasi standar COCO untuk luaran *as-deployed* FP16, dan fungsi validasi bawaan kerangka kerja Ultralytics untuk *baseline* FP32-*proxy*. Perbandingan kedua nilai ini berfungsi sebagai *sanity-check* untuk memastikan tidak ada penurunan akurasi tersembunyi akibat kuantisasi presisi numerik maupun proses penguraian (*parsing*) metadata kustom. Kekhawatiran ini beralasan mengingat *trade-off* antara presisi numerik dan akurasi deteksi pada arsitektur GPU *embedded* telah didokumentasikan pada literatur inferensi terkuantisasi (Guerrouj dkk., 2025).

Metrik efisiensi *tracking* (latensi tahap *tracking* beserta utilisasi GPU/CPU/RAM/daya) sengaja **tidak** disertai metrik kualitas *tracking* seperti *ID switch* atau MOTA/IDF1. Keputusan ini mengikuti preseden metodologis MLPerf Mobile Inference Benchmark (Janapa Reddi dkk., 2022), yang menjadikan akurasi sebagai ambang kelulusan tetap dan bukan variabel yang dibandingkan antar konfigurasi. Rasionalisasi lengkap dapat dilihat pada pembahasan pemilihan *tracker* sebelumnya dan batasan penelitian pada Bab I.

Tabel 2.2 Kriteria evaluasi per rumusan masalah

Sumbu	Metrik	Kriteria
RM1 (<i>baseline</i>)	<i>Throughput</i> keseluruhan	<i>Real-time</i> didefinisikan sebagai throughput ≥ 30 FPS (mengikuti konfigurasi <i>default</i> kamera ZED dan standar ADAS <i>safety-critical perception layer</i> , Costa dkk., 2025)
RM1 (<i>baseline</i>)	Latensi <i>end-to-end</i>	Dilaporkan sebagai rerata dan persentil P95 , bukan hanya rerata sendirian, supaya <i>outlier/jitter</i> (relevan untuk <i>safety-critical</i> , sebagaimana dibahas pada Bab I) tidak tersembunyi
RM2 (NMS)	<i>Throughput</i> & latensi tahap inferensi	Selisih (Δ) <i>baseline</i> vs. EfficientNMS pada pasangan model yang sama; peningkatan dianggap bermakna jika Δ konsisten di seluruh repetisi (bukan kebetulan <i>noise</i> satu percobaan)
RM3 (<i>tracker</i>)	<i>Throughput</i> keseluruhan, latensi tahap <i>tracking</i> , utilisasi GPU/CPU/RAM/daya	Murni efisiensi komputasi , sesuai batasan penelitian pada Bab I. Tidak ada ambang akurasi <i>tracking</i> (<i>ID switch</i> , MOTA/IDF1) yang perlu dipenuhi; konsisten dengan pendekatan MLPerf Mobile Inference Benchmark (Janapa

		Reddi dkk., 2022) yang menjadikan akurasi ambang kelulusan tetap (bukan variabel dibandingkan), sebagaimana dibahas pada pembahasan pemilihan <i>tracker</i>
Akurasi (pendukung, skenario 4)	mAP50, mAP50-95 (<i>as-deployed</i> FP16 vs. <i>proxy</i> FP32)	Bukan variabel dibandingkan antar model, melainkan <i>*kriteria sanity-check pass/fail*</i> : selisih diharapkan kecil (ambang pasti, mis. <1–2 poin, ditentukan penulis berdasarkan referensi literatur umum FP16-vs-FP32 setelah data aktual tersedia)

BAB III: HASIL DAN PEMBAHASAN

3.1 Kondisi Eksekusi dan Parameter Baseline

Pipeline DeepStream berhasil dikompilasi dan dijalankan penuh di Jetson Orin Nano untuk keenam konfigurasi model yang dikombinasikan dengan kedua konfigurasi *tracker*. Lapisan orkestrasi otomatis menjalankan seluruh *12 skenario ($6 \text{ model} \times 2 \text{ tracker}$) $\times 5 \text{ repetisi} = 60 \text{ run}$ ** secara berurutan tanpa intervensi manual di antara run, sesuai tahap keenam metodologi penelitian. Seluruh 60 run berhasil menghasilkan ketiga artefak pencatatan yang diharapkan, yaitu log performa per-frame, log utilisasi perangkat keras, dan metadata proses pengujian, masing-masing pada lokasi tersendiri yang tidak menimpa hasil sebelumnya; tidak ada run* yang gagal atau perlu diulang.

Parameter	Nilai
Versi kode sumber	Konsisten di seluruh 60 run (satu commit tunggal)
Video input	Identik di seluruh run, sesuai variabel terkontrol pada protokol pengujian
Mode keluaran	Berkas (encode MP4)
Mode daya perangkat	10 W, satu-satunya mode performa tinggi yang tersedia pada varian Jetson Orin Nano 4GB yang dipakai; diaktifkan sekali di awal batch sebelum ke-60 run
Penguncian clock perangkat	Dikunci di awal batch dengan mekanisme yang menghentikan seluruh proses pengujian pada run pertama bila langkah penguncian gagal, sehingga status kunci pada 60 run berikutnya terjamin konsisten
Pengukuran latensi	Diaktifkan otomatis di seluruh run

Interval pencuplikan utilitas pemantauan perangkat keras	1.000 milidetik
Rentang waktu eksekusi <i>batch</i>	2026-08-19 11:33–12:46 WITA ($\pm$ 72 menit, termasuk <i>cooldown</i> 60 detik antar- <i>run</i>)

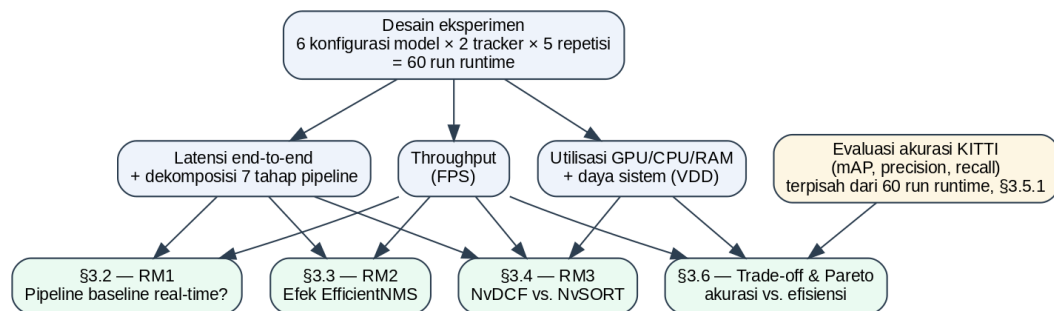
Catatan metodologis: metadata pengujian pada masing-masing *run* mencatat galat izin akses pada kolom status penguncian *clock*. Ini **bukan** indikasi bahwa *clock* tidak terkunci: galat ini berasal dari pemeriksaan status yang berjalan sebagai proses anak tanpa privilese administratif, terpisah dari langkah pengunci *clock* yang sesungguhnya (dijalankan sekali di awal orkestrasi *batch* dengan privilese administratif penuh). Karena mekanisme orkestrasi otomatis menghentikan seluruh proses pada kegagalan langkah manapun, kegagalan pada langkah pengunci itu sendiri akan menghentikan seluruh *batch* di *run* pertama, sehingga 60 *run* yang berhasil selesai secara konsisten mengindikasikan langkah penguncian berhasil. Kelemahan pencatatan status ini (bukan kelemahan pengujian) dicantumkan sebagai catatan transparansi, bukan disembunyikan.

Durasi setiap *run* individual relatif singkat ($\pm$ 13–20 detik *wall-clock*, mengikuti panjang video input) karena berkas input diputar apa adanya tanpa batas durasi eksplisit yang dikonfigurasi. Agregasi membuang 10 detik pertama setiap *run* sebagai *warm-up* (mengikuti rekomendasi protokol pengujian), sehingga jumlah *frame* yang tersisa untuk statistik per skenario berkisar 633–1.399 *frame* (gabungan 5 repetisi), lebih sedikit dibanding rekomendasi klip 180 detik pada protokol. Implikasi keterbatasan ini dibahas lebih lanjut pada pembahasan keterbatasan sistem di akhir bab ini.

Data mentah 60 *run* diagregasi menjadi ringkasan per skenario (rerata antar-5-repetisi) dan data mentah per *run* (dipakai untuk uji signifikansi pada pembahasan

optimasi NMS dan efisiensi *tracking*), ditambah grafik distribusi FPS per model dan per *tracker*. Data ini kemudian digabung dengan hasil akurasi KITTI menjadi rangkuman *trade-off* dan grafik Pareto, dipakai pada pembahasan akhir bab ini.

Diagram berikut memetakan **struktur pengukuran** bab ini: dari desain eksperimen, tiga kelompok metrik *runtime* yang dicatat per *run*, hingga rumusan masalah (RM) mana yang dijawab tiap kelompok metrik pada subbab berikutnya, bukan sekadar mengulang alur agregasi data di atas.



Struktur pengukuran: dari desain eksperimen menuju kelompok metrik runtime dan rumusan masalah yang dijawab.

Peta ini menegaskan bahwa akurasi KITTI **tidak** berasal dari 60 *run* pengujian *runtime* di atas: keduanya adalah dua sumber data independen yang baru digabung pada tahap akhir untuk analisis *trade-off* pada pembahasan akhir bab ini, bukan bagian dari pipa agregasi *runtime* yang sama.

3.2 Hasil Pengujian Kinerja Baseline Pipeline (Menjawab RM1)

3.2.1 Analisis Throughput (FPS)

Skenario *baseline* memakai *tracker default* NvDCF, sesuai skenario pengujian pertama pada protokol metodologi penelitian.

Model	Avg FPS	Median FPS	Std FPS	Ambang (vs. 30 FPS)
YOLOv8n	66,77	66,52	0,51	+123%

YOLOv9t	51,52	51,73	0,61	+72%
YOLOv10n	67,02	67,22	0,40	+123%
YOLO26n	65,66	65,45	1,02	+119%

Seluruh model baseline jauh melampaui ambang real-time 30 FPS, bahkan YOLOv9t yang paling lambat sekalipun mencapai rerata 51,52 FPS ($\pm 72\%$ di atas ambang). Pemenuhan ambang ini juga diverifikasi tidak hanya pada rerata, tetapi pada setiap dari 60 run individual (bukan hanya rerata skenario, agar run tunggal yang kebetulan lambat tidak tersembunyi di balik rerata):*

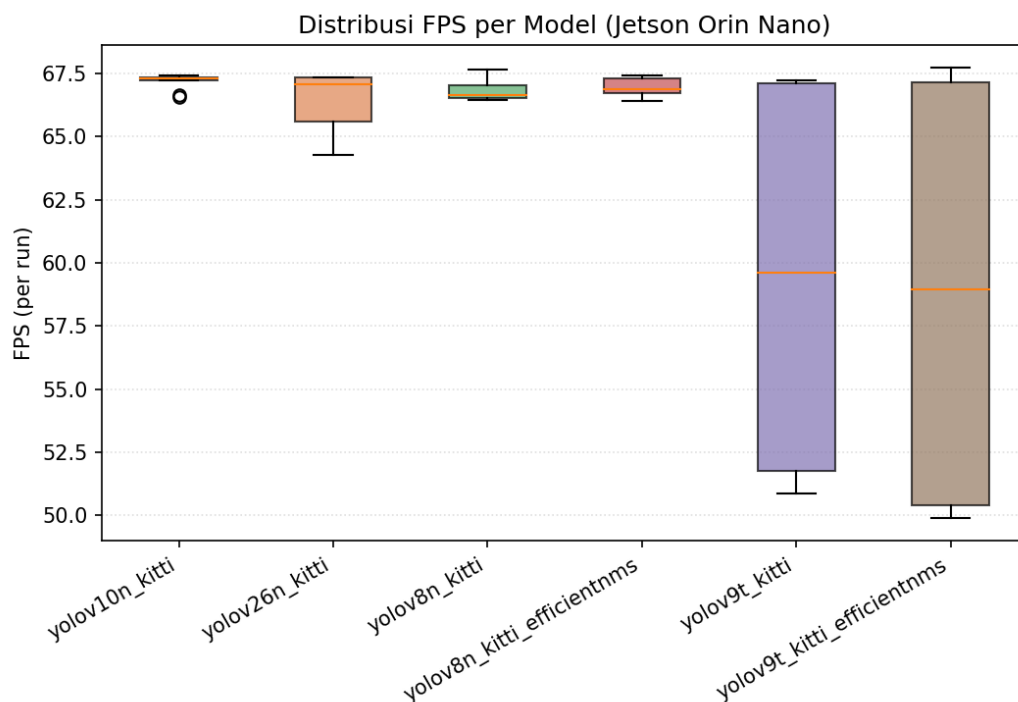
Model	Tracker	FPS minimum (dari 5 repetisi)	FPS maksimum	Status vs. ambang 30 FPS
YOLOv8n	NvDCF	66,43	67,66	Lulus (+121%)
YOLOv8n	NvSORT	66,49	67,41	Lulus (+122%)
YOLOv9t	NvDCF	50,88	52,29	Lulus (+70%)
YOLOv9t	NvSORT	66,90	67,23	Lulus (+123%)
YOLOv10n	NvDCF	66,55	67,40	Lulus (+122%)
YOLOv10n	NvSORT	67,31	67,37	Lulus (+124%)
YOLO26n	NvDCF	64,27	67,06	Lulus (+114%)
YOLO26n	NvSORT	67,08	67,34	Lulus (+124%)
YOLOv8n+EfficientNMS	NvDCF	66,42	66,82	Lulus (+121%)
YOLOv8n+EfficientNMS	NvSORT	66,94	67,43	Lulus (+123%)
YOLOv9t+EfficientNMS	NvDCF	49,89	51,18	Lulus (+66%)

ientNMS				
YOLOv9t+EfficientNMS	NvSORT	66,75	67,73	Lulus (+123%)

Seluruh 60 run, tanpa kecuali, memenuhi kriteria real-time ≥ 30 FPS, bahkan skenario paling lambat (YOLOv9t+EfficientNMS dengan NvDCF, FPS minimum 49,89) masih melampaui ambang sebesar 66%. Ini menjawab bagian inti RM1: keempat model baseline memenuhi kriteria real-time pada Jetson Orin Nano dengan margin yang besar, sehingga margin tersebut dapat "dikonversi" untuk mengejar akurasi lebih tinggi pada pembahasan trade-off di akhir bab ini tanpa mengorbankan kepatuhan real-time. Rincian FPS pada varian EfficientNMS dan varian tracker NvSORT dibahas lebih lanjut masing-masing pada subbab optimasi NMS paralel dan subbab efisiensi tracking*.

Catatan lingkup pengujian: hasil di atas berasal dari protokol berbasis berkas video terkontrol menggunakan satu video input yang identik di seluruh run, bukan dari kamera ZED live secara langsung. Keputusan ini disengaja (sumber video tetap agar setiap model "melihat" input yang identik, sehingga hasil antar-model benar-benar sebanding). Validasi tambahan pada aliran kamera ZED live belum dilakukan pada laporan ini dan dicatat sebagai pekerjaan lanjutan pada pembahasan keterbatasan sistem.

Gambar 3.1 Distribusi FPS per model (seluruh run, kedua konfigurasi tracker digabung)



Distribusi FPS per model, digambarkan sebagai kotak-garis (boxplot) yang merangkum rentang, median, dan sebaran seluruh run per model

Grafik di atas memvisualkan sebaran FPS pada Tabel di atas dalam bentuk kotak-garis: keempat model *baseline* tampak terkumpul rapat di kisaran 65–68 FPS, sementara YOLOv9t menunjukkan sebaran dua-modus (*bimodal*) yang jauh lebih lebar, konsekuensi langsung dari interaksi model-*tracker* yang dibahas lebih rinci pada subbab dampak algoritma *tracking* terhadap FPS.

3.2.2 Analisis Latensi End-to-End

Model	Average Latensi (ms)	Average P95 Latensi (ms)
YOLOv8n	273,35	329,10
YOLOv9t	457,67	536,32
YOLOv10n	247,48	299,46
YOLO26n	355,56	409,20

--	--	--

Latensi *end-to-end* pada Tabel di atas sengaja dilaporkan sebagai rerata **P95** selain rerata keseluruhan, mengikuti kriteria evaluasi RM1, agar *outlier/jitter* (relevan untuk aplikasi *safety-critical* seperti ADAS) tidak tersembunyi di balik rerata. **YOLOv9t adalah outlier yang konsisten dengan temuan akurasi pada evaluasi GPU cloud** yang mencatat YOLOv9t sebagai model dengan waktu inferensi paling lambat pada GPU cloud walau parameternya paling kecil. Di Jetson, pola ini terulang dan makin nyata pada sisi latensi: rerata 457,67 ms dan rerata P95 536,32 ms, jauh di atas ketiga model lain (rerata 247,48–355,56 ms, rerata P95 299,46–409,20 ms). Selisih antara rerata dan P95 pada YOLOv9t (~78,7 ms) juga lebih besar dibanding model lain (~52–56 ms), mengindikasikan sebaran jitter yang lebih lebar pada model ini, sebuah pertimbangan penting jika YOLOv9t hendak dipakai pada sistem safety-critical* yang menuntut latensi terprediksi, bukan sekadar rerata rendah.*

3.2.3 Dekomposisi Latensi Per-Komponen

Rincian latensi rerata per-komponen *pipeline* mengurai kontribusi tiap tahap terhadap total latensi pada tabel analisis latensi *end-to-end* sebelumnya:

Model	Pra-multiplexing	Multiple xing	Inferensi	Tracking	Pra-OSD	OSD	Output
YOLOv8n	213,84	14,09	19,21	10,98	~0,01	9,34	5,87
YOLOv9t	360,92	27,48	28,65	18,93	~0,01	13,76	7,93

YOLOv10n	205,36	10,11	16,72	3,53	~0,01	7,08	4,67
YOLO26n	279,70	21,87	22,88	13,63	~0,01	10,95	6,52

Gambar 3.2 Dekomposisi latensi per-komponen, model *baseline* dengan *tracker* NvDCF

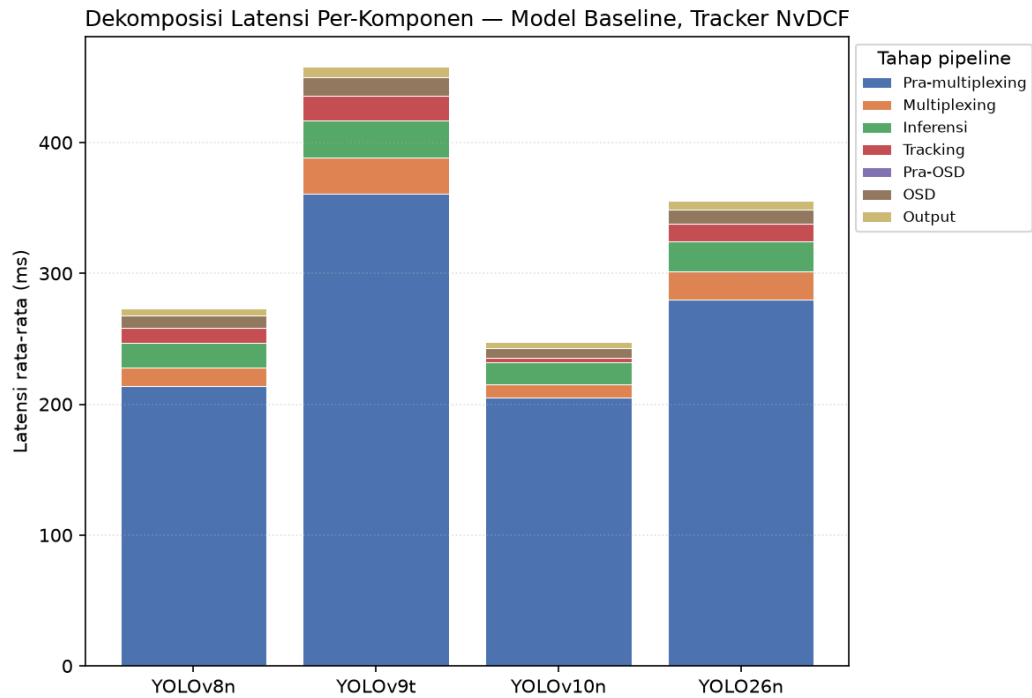


Diagram batang bertumpuk dekomposisi latensi tujuh tahap pipeline per model baseline

Diagram batang bertumpuk di atas memvisualkan proporsi Tabel di atas: tahap *pra-multiplexing* secara konsisten mendominasi total latensi pada seluruh model, dengan kontribusinya pada YOLOv9t yang tampak jelas paling besar secara absolut dibanding ketiga model lain.

Dua temuan utama menjelaskan *bottleneck* per-komponen:

35. *Latensi tahap pra-multiplexing YOLOv9t (360,92 ms) jauh di atas model lain (~205–280 ms). Tahap ini mengukur waktu tunggu buffer di decoder/antrean sebelum masuk ke tahap multiplexing aliran video; nilainya yang membengkak untuk YOLOv9t kemungkinan besar adalah gejala backpressure dari tahap hilir yang lebih lambat (arsitektur PGI/GELAN YOLOv9 yang lebih sequential, ditambah biaya NvDCF di tahap tracking) yang merambat balik ke antrean di depan tahap multiplexing, bukan decoder itu sendiri yang melambat. Dugaan ini konsisten dengan pola pada perbandingan latensi tracker (latensi pra-multiplexing YOLOv9t turun ke ~206 ms, sejajar model lain, begitu tracker* diganti ke NvSORT).*
36. **YOLO26n memiliki GFLOPs teoretis terendah (5,2) tetapi bukan model tercepat di Jetson (65,66 FPS, di bawah YOLOv8n dan YOLOv10n),** mengonfirmasi temuan pada evaluasi akurasi bahwa GFLOPs teoretis tidak selalu berbanding lurus dengan performa aktual pada perangkat *edge*; kepadatan komputasi memori/latensi *tracker* NvDCF (13,63 ms untuk YOLO26n, kedua tertinggi setelah YOLOv9t) turut menyumbang. Diskusi lanjutan tentang karakter NMS-free YOLO26n ada pada subbab pembahasan model NMS-free.

3.3 Hasil Pengujian Optimasi NMS Paralel (Menjawab RM2)

Perbandingan pasangan model dengan bobot identik, *tracker* NvDCF (kondisi lain sama dengan pengujian kinerja *baseline* sebelumnya), sesuai skenario pengujian kedua pada protokol metodologi penelitian. Uji signifikansi memakai Welch's *t*-test atas distribusi FPS 5 repetisi per skenario.

3.3.1 Dampak EfficientNMS terhadap Latensi Inferensi

Pasangan	Latensi inferensi <i>baseline</i> (ms)	Latensi inferensi EfficientNMS (ms)	Δ
YOLOv8n	19,21	18,97	-0,24 ms

YOLOv9t	28,65	28,09	-0,56 ms
---------	-------	-------	----------

Pada kedua model, latensi tahap inferensi (yang mencakup *post-processing* NMS) sedikit membaik setelah EfficientNMS dipasang, namun selisihnya sangat kecil secara absolut. Ini konsisten dengan pengukuran mandiri terhadap biaya operasi NMS paralel berbasis *plugin* itu sendiri, yang menunjukkan biayanya sudah sangat murah (~0,05 ms pada TensorRT 10.3 untuk YOLOv8n *batch* 1): implementasi NMS standar yang dipakai *baseline* bukanlah *bottleneck* besar sejak awal, sehingga ada sedikit "ruang" yang bisa dioptimasi pada tahap inferensi itu sendiri.

3.3.2 Dampak EfficientNMS terhadap Throughput Keseluruhan

Pasangan	FPS <i>baseline</i>	FPS EfficientNMS	Δ FPS	<i>p</i> -value	Kesimpulan
YOLOv8n vs. +EfficientNMS	66,77	66,63	-0,14 (-0,2%)	0,592	Tidak signifikan
YOLOv9t vs. +EfficientNMS	51,52	50,44	-1,08 (-2,1%)	0,023	Signifikan, EfficientNMS lebih lambat

Latensi total rerata (ms): YOLOv8n 273,35 ms (*baseline*) vs. 262,34 ms (EfficientNMS); YOLOv9t 457,67 ms (*baseline*) vs. 468,26 ms (EfficientNMS).

Pada YOLOv8n, tidak ada selisih FPS yang signifikan secara statistik. Pada YOLOv9t, sebaliknya, selisih FPS justru **signifikan namun berlawanan arah dari yang diharapkan**: EfficientNMS lebih lambat, bukan lebih cepat. Karena latensi

tahap inferensi itu sendiri hampir tidak berubah, penurunan FPS pada YOLOv9t bukan berasal dari biaya operasi NMS paralel itu sendiri, melainkan dari interaksi dengan tahap lain. Hasil ini adalah **temuan negatif yang sah secara ilmiah**, bukan kegagalan implementasi, dan dapat dijelaskan oleh tiga faktor:

37. **Biaya operasi NMS paralel itu sendiri sudah sangat kecil**, sehingga ruang

optimasi dari sisi *plugin* NMS memang terbatas sejak awal.

38. *Operasi NMS paralel adalah tail yang dependen pada output detektor: ia tidak berjalan bersamaan (concurrent) dengan komputasi backbone untuk frame yang sama. Karakteristik sequential/tidak-overlapping ini kemungkinan mengurangi kesempatan pipelining GPU antar frame, yang konsisten dengan turunnya rerata pemakaian GPU pada YOLOv9t+EfficientNMS (69,3%, turun dari 87,7% pada baseline, lihat pembahasan analisis penggunaan sumber daya perangkat) meski FPS-nya juga turun, indikasi bubble/idle* GPU yang lebih besar, bukan GPU yang lebih sibuk.*

39. **Perbaikan latensi inferensi semata tidak otomatis menaikkan FPS** bila tahap lain (tahap *tracking* atau tahap *pra-multiplexing*) menjadi *bottleneck* yang lebih dominan, persis kondisi YOLOv9t pada dekomposisi latensi per-komponen, di mana latensi inferensi nyaris tidak berubah tetapi FPS turun karena interaksi dengan *tail latency* di tahap lain.

Peluang optimasi lanjutan yang lebih menjanjikan untuk kasus ini bukan pada *plugin* NMS paralel itu sendiri, melainkan pada penyesuaian ambang keyakinan (*confidence*) dan batas jumlah *bounding box* keluaran yang lebih agresif (bila mAP masih dalam toleransi) atau model dengan *head* NMS-free, dibahas pada subbab berikut.

3.3.3 Pembahasan Model NMS-free

YOLOv10n dan YOLO26n tidak disertakan pada perbandingan RM2 di atas karena keduanya sudah *NMS-free* secara arsitektural, sehingga tidak ada pasangan

baseline vs. EfficientNMS yang setara untuk dibandingkan. Namun demikian, hasil *baseline* pada analisis *throughput* dan dekomposisi latensi sebelumnya tetap relevan untuk menilai apakah pendekatan arsitektural NMS-free memberi keuntungan runtime dibanding model dengan NMS terpisah: YOLOv10n mencatat FPS tertinggi di antara keempat model *baseline* (67,02) dengan latensi inferensi terendah (16,72 ms), sedangkan YOLO26n, meski memiliki GFLOPs teoretis terendah (5,2), hanya mencapai 65,66 FPS, di bawah YOLOv8n dan YOLOv10n. Dengan kata lain, *karakter NMS-free tidak secara otomatis menjamin throughput tertinggi pada level pipeline lengkap: faktor lain seperti kepadatan komputasi backbone dan biaya tracker* NvDCF tetap berkontribusi signifikan terhadap FPS akhir.

Temuan ini melengkapi hasil RM2: bagi model kelas *nano/tiny* pada Jetson Orin Nano, pendekatan menghilangkan NMS secara arsitektural (seperti pada YOLOv10n/YOLO26n) dan pendekatan mempercepat NMS lewat *plugin* paralel (EfficientNMS pada YOLOv8n/YOLOv9t) sama-sama **tidak** memberi jaminan otomatis atas peningkatan *throughput* yang besar. Pada kelas model ini, latensi tahap inferensi (termasuk NMS di dalamnya) bukan komponen *bottleneck* dominan dibanding latensi tahap *pre-multiplexing* dan tahap *tracking*, sebagaimana ditunjukkan pada dekomposisi latensi dan perbandingan latensi *tracker*.

3.4 Hasil Pengujian Efisiensi Komputasi Tracking (Menjawab RM3)

Skenario ini adalah inti rumusan masalah #3 (skenario pengujian ketiga pada protokol metodologi penelitian): membandingkan **efisiensi komputasi** (bukan kualitas *tracking*) NvDCF vs. NvSORT di keenam model, masing-masing 5 repetisi (total 60 *run*, lihat bagian pembuka bab ini). Statistik latensi tahap *tracking* di bawah dihitung sebagai rerata antar-5-repetisi per skenario, konsisten dengan cara agregasi yang sama dipakai pada seluruh tabel latensi lain di bab ini.

3.4.1 Perbandingan Latensi Tracker (NvDCF vs. NvSORT)

Tabel 3.4.1 Latensi tahap *tracking* rerata (antar-5-repetisi per skenario)

Model	NvDCF (ms)	NvSORT (ms)	Rasio Percepatan (NvDCF/NvSORT)
YOLOv8n	10,981	0,363	~30×
YOLOv9t	18,935	0,723	~26×
YOLOv10n	3,530	0,297	~12×
YOLO26n	13,633	0,428	~32×
YOLOv8n+Efficient NMS	11,650	0,401	~29×
YOLOv9t+Efficient NMS	20,787	0,644	~32×

Gambar 3.4 Latensi tahap *tracking* rerata: NvDCF vs. NvSORT (skala logaritmik)

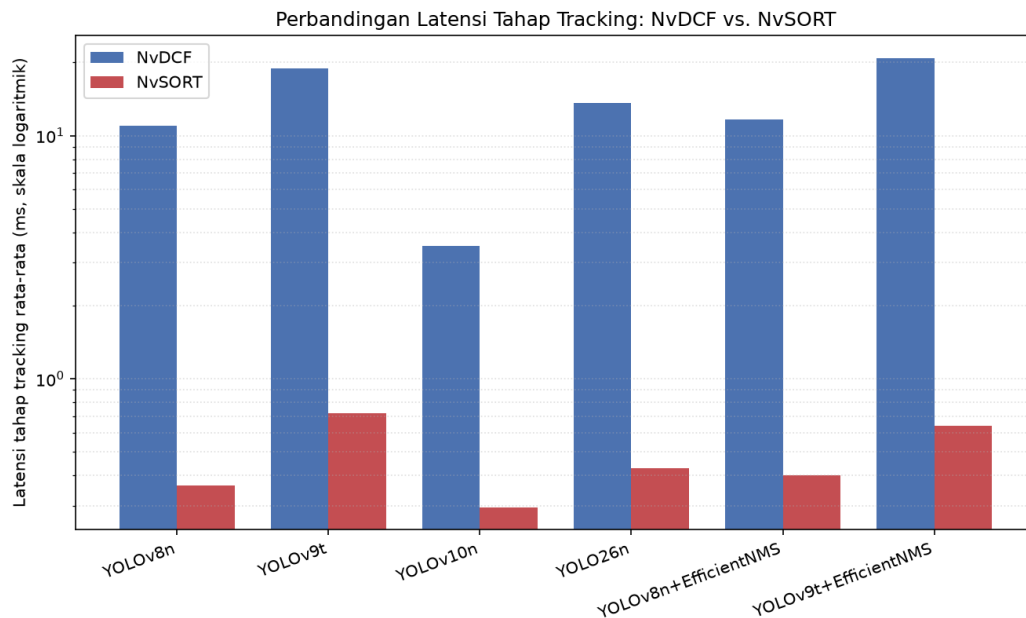


Diagram batang berkelompok latensi tahap tracking NvDCF dibandingkan NvSORT pada skala logaritmik, keenam model

Skala logaritmik pada sumbu-y sengaja dipakai agar selisih puluhan-kali-lipat pada Tabel di atas tetap terbaca proporsional dalam satu grafik: batang NvSORT tampak nyaris rata di dasar grafik pada skala linear biasa, padahal nilainya tetap bervariasi antar model.

Biaya komputasi latensi tahap tracking NvDCF secara konsisten $\sim 12\times$ – $32\times$ lebih tinggi daripada NvSORT, di seluruh enam model, sejalan dengan karakteristik arsitektural keduanya: NvDCF melakukan ekstraksi fitur berbasis piksel penuh per objek, sedangkan NvSORT murni berbasis gerak (Kalman filter + algoritma Hungarian) tanpa pemrosesan piksel. Ini adalah hasil yang tidak bergantung pada model deteksi: pola yang sama muncul baik pada model nano-class ringan (YOLOv10n, rasio terendah $\sim 12\times$) maupun yang lebih berat (YOLOv9t/YOLO26n, rasio ~ 26 – $32\times$); rasio terendah pada YOLOv10n konsisten dengan biaya latensi tracking NvDCF absolutnya yang juga paling rendah (3,530 ms) di antara keenam model.*

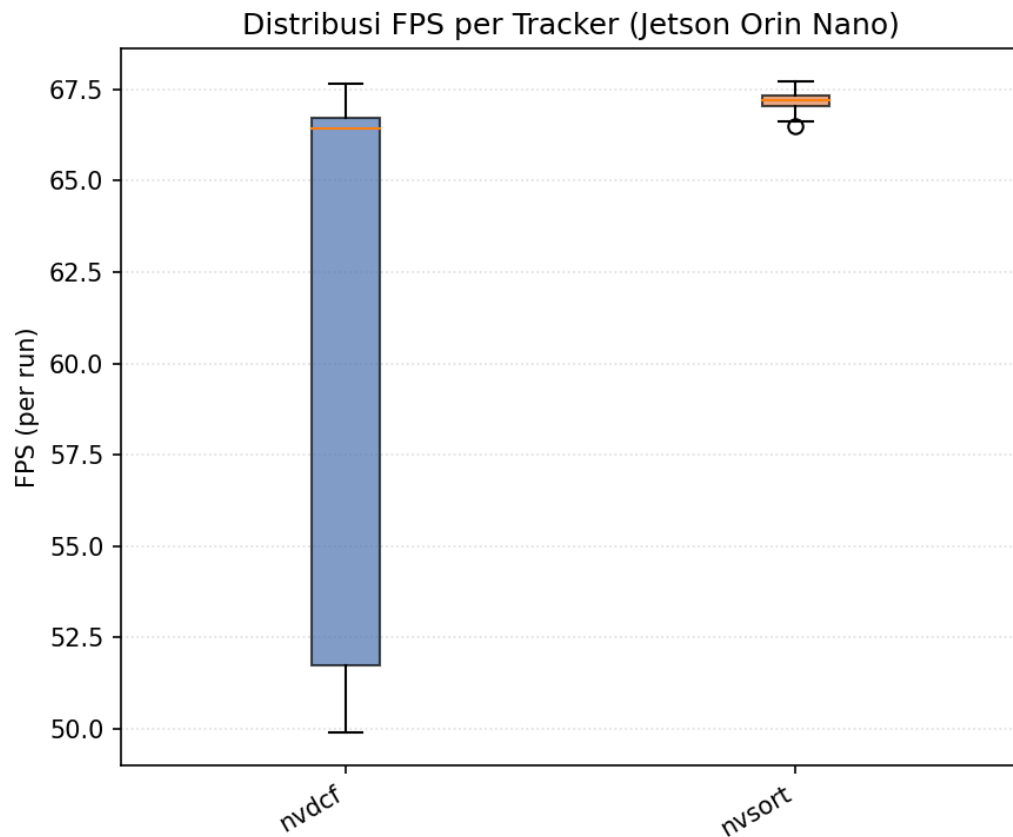
3.4.2 Dampak Algoritma Tracking terhadap FPS

Tabel 3.4.2 FPS dan signifikansi statistik (Welch's *t*-test, NvDCF vs. NvSORT per model)

Model	FPS NvDCF	FPS NvSORT	Δ FPS	<i>p</i> -value	Signifikan?
YOLOv8n	66,77	66,86	+0,09	0,753	Tidak
YOLOv9t	51,52	67,08	+15,56 (+30%)	<0,0001	Ya (sangat kuat)
YOLOv10n	67,02	67,34	+0,32	0,146	Tidak
YOLO26n	65,66	67,29	+1,63	0,023	Ya
YOLOv8n+E fficientNMS	66,63	67,23	+0,60	0,003	Ya
YOLOv9t+E fficientNMS	50,44	67,19	+16,75 (+33%)	<0,0001	Ya (sangat kuat)

Pola ini juga tergambar pada distribusi FPS per *tracker* di seluruh skenario.

Gambar 3.3 Distribusi FPS per *tracker* (seluruh *run*, keenam model digabung)



Distribusi FPS per tracker, digambarkan sebagai kotak-garis yang membandingkan sebaran NvDCF dan NvSORT

Sebaran NvDCF pada grafik di atas jauh lebih lebar dibanding NvSORT yang nyaris tanpa variasi, cerminan langsung dari kontribusi YOLOv9t yang menarik sebaran NvDCF ke bawah, sementara skenario NvSORT untuk seluruh model (termasuk YOLOv9t) tetap terkumpul rapat di atas 66 FPS.

Dampaknya terhadap FPS keseluruhan bergantung pada model: inilah temuan paling penting dari RM3. Untuk YOLOv8n, YOLOv10n, dan (secara praktis) YOLO26n/YOLOv8n+EfficientNMS, selisih FPS akibat pergantian *tracker* kecil (0,09–1,63 FPS) meski sebagian secara statistik signifikan (karena variansi antar-*run* yang sangat kecil, bukan karena selisihnya besar secara praktis). Sebaliknya, untuk *YOLOv9t (baik varian baseline maupun EfficientNMS), pergantian

NvDCF → NvSORT meningkatkan FPS +30% dan +33%, selisih yang sangat besar dan sangat signifikan ($p^ < 0,0001$).*

Temuan inti RM3 dapat diringkas sebagai *"penghematan komputasi NvSORT bersifat universal pada level komponen, tetapi manfaatnya pada throughput akhir bersifat kondisional pada model."* Latensi tahap tracking NvSORT konsisten mendekati nol (rerata $< 0,8$ ms) untuk keenam model, properti arsitektural NvSORT itu sendiri yang tidak bergantung pada model deteksi di hulu. Namun demikian, penghematan komputasi ini hanya "terlihat" pada throughput akhir ketika model deteksi di hulu sudah menghabiskan sebagian besar headroom waktu-per-frame yang tersedia, kondisi yang secara empiris hanya terpenuhi oleh YOLOv9t pada eksperimen ini (gabungan latensi tahap inferensi, pra-multiplexing, dan tracking mendekati/melebihi waktu per-frame yang dibutuhkan untuk mencapai 60 FPS, lihat pembahasan dekomposisi latensi per-komponen). Pada YOLOv8n, YOLOv10n, dan YOLO26n, yang memiliki headroom lebih besar, biaya tambahan NvDCF "tersembunyi" di dalam slack tersebut dan tidak sampai menjadi bottleneck akhir pipeline*.

Implikasi praktis untuk deployment ADAS: **pemilihan tracker tidak dapat dievaluasi secara independen dari model deteksi yang dipasangkan dengannya. Rekomendasi generik "gunakan NvSORT karena lebih efisien" benar secara komponen, tetapi dampaknya terhadap throughput keseluruhan sistem baru signifikan pada kombinasi model yang pipeline-nya sudah mendekati batas kapasitas, sesuatu yang hanya dapat diketahui melalui pengukuran end-to-end seperti yang dilakukan pada bab ini, bukan dari spesifikasi tracker* semata.*

3.4.3 Analisis Penggunaan Sumber Daya Perangkat

Tabel 3.4.3 Efisiensi perangkat keras (rerata GPU%, RAM, daya sistem total, dan estimasi energi per *frame* = daya sistem total ÷ FPS)

Model	Tracker	GPU %	RAM (MB)	Daya sistem	Energi/
-------	---------	-------	----------	-------------	---------

				total (mW)	<i>frame</i> (mJ)
YOLOv8n	NvDCF	84,6	1368,0	9185,8	137,58
YOLOv8n	NvSORT	64,3	1296,5	8447,3	126,34
YOLOv9t	NvDCF	87,7	1365,2	8598,3	166,89
YOLOv9t	NvSORT	90,6	1300,5	8930,2	133,12
YOLOv10n	NvDCF	66,9	1449,1	8614,0	128,53
YOLOv10n	NvSORT	65,6	1388,4	8569,2	127,26
YOLO26n	NvDCF	75,6	1418,0	8827,6	134,45
YOLO26n	NvSORT	68,1	1389,5	8455,5	125,66
YOLOv8n+E efficientNMS	NvDCF	87,2	1371,3	9202,0	138,11
YOLOv8n+E efficientNMS	NvSORT	57,4	1310,7	8443,2	125,58
YOLOv9t+E efficientNMS	NvDCF	69,3	1361,7	8389,2	166,32
YOLOv9t+E efficientNMS	NvSORT	91,2	1318,2	8942,3	133,10

Gambar 3.5 Estimasi energi per-*frame*: NvDCF vs. NvSORT

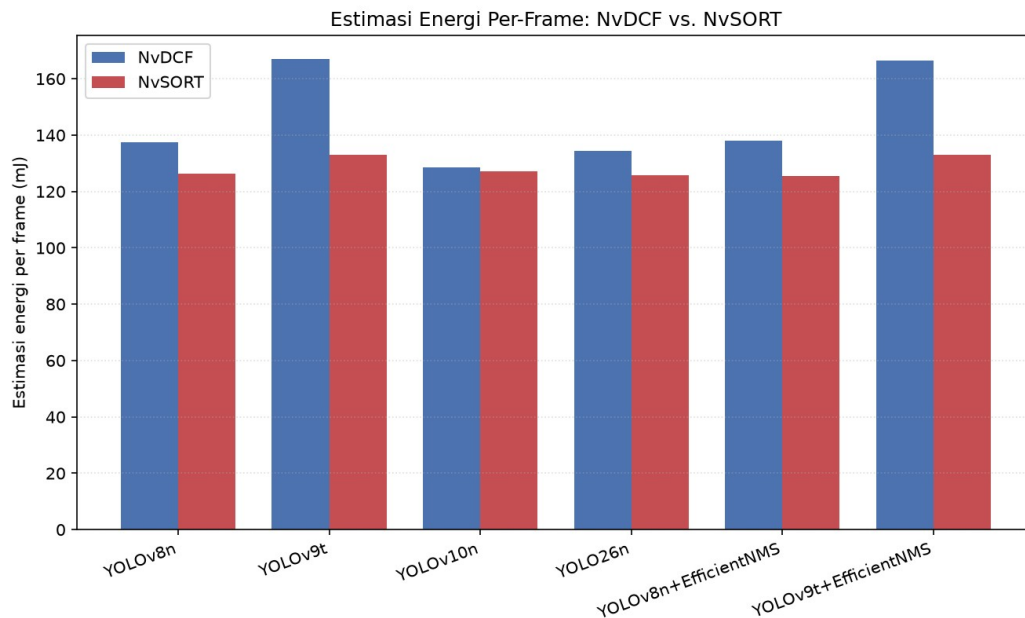


Diagram batang berkelompok estimasi energi per frame NvDCF dibandingkan NvSORT, keenam model

Grafik di atas menegaskan pola pada Tabel di atas: batang NvSORT konsisten sama tinggi atau lebih rendah daripada NvDCF di seluruh model, dengan selisih paling mencolok pada kedua varian YOLOv9t.

Dua temuan tambahan dari sisi *resource*:

40. *Estimasi energi per frame (daya ÷ FPS) menunjukkan NvSORT lebih hemat energi di seluruh enam model, walau daya sesaat sistem total NvSORT justru sedikit lebih tinggi daripada NvDCF pada kedua varian YOLOv9t (8930,2 vs. 8598,3 mW dan 8942,3 vs. 8389,2 mW). Ini karena NvSORT menyelesaikan jauh lebih banyak frame per detik pada YOLOv9t (67 vs. 51 FPS), sehingga energi yang dikeluarkan per unit pekerjaan (per frame) tetap lebih rendah (133 mJ vs. 167 mJ) meski daya sesaatnya lebih tinggi. Metrik daya sesaat sendirian dapat menyesatkan bila tidak dinormalisasi terhadap throughput*; inilah alasan bagian ini melaporkan kedua metrik.*

41. **Pola GPU% pada YOLOv9t terbalik dibanding empat model lain:** pada YOLOv8n/YOLOv10n/YOLO26n, NvDCF menghasilkan GPU% lebih tinggi daripada NvSORT (selisih 1,3–29,8 poin), sedangkan pada YOLOv9t, NvSORT justru menghasilkan GPU% *lebih tinggi* (90,6% vs. 87,7% dan 91,2% vs. 69,3%). Kemungkinan penjelasannya: pada YOLOv9t+NvSORT, hilangnya *bottleneck tracker* membuat GPU bekerja mendekati kapasitas penuhnya untuk inferensi (GPU-bound), sedangkan pada YOLOv9t+NvDCF, sebagian waktu justru dihabiskan menunggu komputasi NvDCF (yang sebagian berjalan di CPU) tanpa GPU *idle* sepenuhnya tercatat sebagai penurunan besar; sampel pemantauan perangkat keras pada interval 1 Hz terlalu kasar untuk memastikan mekanisme persisnya. Pola ini dilaporkan sebagai temuan yang memerlukan profil lebih dalam menggunakan perangkat pembuat profil (*profiler*) tingkat rendah GPU, bukan disimpulkan secara pasti pada bagian ini.

Penggunaan RAM berkisar 1.265–1.449 MB di seluruh 12 konfigurasi, sekitar 31–35% dari kapasitas total 4GB modul Jetson Orin Nano yang dipakai untuk satu *stream* video dan satu model aktif. Implikasi keterbatasan memori ini untuk skenario *deployment* yang lebih kompleks (mis. multi-kamera/multi-model) dibahas pada pembahasan keterbatasan sistem.

Kualitas/akurasi tracking (ID switch, MOTA/IDF1) sengaja tidak diukur pada bagian ini, sesuai batasan lingkup penelitian dan justifikasi pemilihan tracker pada bab metode penelitian; perbandingan di atas murni efisiensi komputasi.*

3.5 Verifikasi Akurasi As-Deployed FP16 (Uji Sanity Check)

3.5.1 Evaluasi Nilai mAP50 dan mAP50-95 (Baseline Proxy FP32)

Sebagai rujukan akurasi, tabel berikut diukur menggunakan fungsi validasi bawaan kerangka kerja Ultralytics di GPU *cloud* (Tesla T4, Kaggle) pada *val set* KITTI yang identik untuk keempat arsitektur dasar (1.010 gambar, 4.722 *instance*).

Nilai ini berlaku untuk pasangan *baseline*/EfficientNMS yang memakai bobot sama (EfficientNMS hanya mengubah eksekusi NMS, bukan bobot deteksi).

Model	Params	GFLOPs	mAP50	mAP50-95	Precision	Recall
YOLOv8n	3.006.233	8,1	0,9767	0,8397	0,9696	0,9344
YOLOv9t	1.971.369	7,6	0,9670	0,8120	0,9643	0,9259
YOLOv10n	2.265.753	6,5	0,9704	0,8370	0,9689	0,9189
YOLO26n	2.375.421	5,2	0,9706	0,8233	0,9508	0,9297

(**Tebal** = nilai terbaik pada kolom tersebut.) YOLOv8n memimpin di mAP50-95 dan *precision*, tetapi selisihnya terhadap YOLOv10n hanya 0,3 poin mAP50-95 meski YOLOv10n memakai ~20% lebih sedikit GFLOPs; temuan ini menjadi salah satu dasar analisis *trade-off* pada pembahasan akhir bab ini. Diskusi per-kelas dan temuan detail lain tidak diulang di sini untuk menghindari duplikasi.

Nilai mAP50/mAP50-95 as-deployed FP16: *TODO: belum dieksekusi*. Tabel yang seharusnya menyandingkan mAP50/mAP50-95 luaran pipeline DeepStream FP16 sesungguhnya (dihitung dari metadata deteksi pipeline* menggunakan pustaka evaluasi standar COCO) akan mengisi bagian ini setelah eksekusi lapangan selesai (lihat status implementasi lengkap pada subbab analisis deviasi akurasi berikut). Angka tidak diisi dengan perkiraan agar tidak melanggar aturan anti-karangan data.

3.5.2 Analisis Deviasi Akurasi FP16 vs. Proxy FP32

TODO: belum dieksekusi. Infrastrukturnya sudah selesai diimplementasikan (lihat skenario pengujian keempat pada protokol metodologi penelitian), tetapi langkah eksekusi nyata di Jetson (ekspor 1.010 gambar val ke perangkat, jalankan pencatatan deteksi FP16, hitung mAP, bandingkan dengan hasil evaluasi mAP50/mAP50-95 pada subbab sebelumnya) **belum dilakukan** pada 60 *run* yang

dilaporkan di bab ini; *run* tersebut memakai video pengujian tetap, bukan video hasil ekspor gambar val KITTI. Tabel deviasi (Δ mAP50, Δ mAP50-95, FP32 vs. FP16) akan diisi setelah seluruh langkah pada skenario pengujian keempat tersebut selesai.

Sebagai kriteria keberhasilan yang sudah ditetapkan lebih dahulu, deviasi ini akan dinilai sebagai **pass/fail** *sanity check*, bukan variabel yang dibandingkan antar model, dengan tujuan membuktikan bahwa optimasi performa komputasi yang dibahas pada subbab-subbab sebelumnya tidak mengorbankan akurasi deteksi di luar batas toleransi yang wajar untuk kuantisasi FP16 pada model YOLO.

3.6 Pembahasan Akhir dan Analisis Trade-Off

3.6.1 Kompromi Kecepatan, Akurasi, dan Efisiensi Energi

Menggabungkan hasil evaluasi akurasi KITTI dan hasil pengujian *runtime*/perangkat keras dari subbab-subbab sebelumnya, tergambar pada diagram pasangan (*dumbbell*) akurasi-vs-*throughput* dan akurasi-vs-daya, seluruh model *baseline* melampaui ambang *real-time* dengan margin besar, sehingga tidak ada model yang "gugur" murni karena kecepatan. Karena itu, rekomendasi disusun berkondisi, bukan klaim satu model "terbaik" tunggal. Kedua diagram menyusun **satu baris per model** (diurutkan menurut mAP50-95, akurasi tertinggi di baris paling atas; akurasi identik untuk kedua *tracker* pada model yang sama, lihat catatan Tabel 3.5.1) alih-alih menggabungkan kedua belas kombinasi model×*tracker* sebagai titik-titik lepas: titik NvDCF dan NvSORT pada tiap baris dihubungkan garis yang panjangnya *langsung* menunjukkan selisih akibat pergantian *tracker*, dengan nilai selisihnya dicetak di atas garis. Bentuk ini dipilih karena akurasi tidak berubah antar-*tracker*, sehingga menaruh kedua belas kombinasi pada satu sumbu-y kontinu (seperti pada draf sebelumnya) hanya menumpuk titik yang sebenarnya berbagi nilai-y persis sama.

Gambar 3.6 *Trade-off* akurasi vs. kecepatan, satu baris per model, NvDCF (biru) vs. NvSORT (merah) dihubungkan garis berlabel selisih FPS

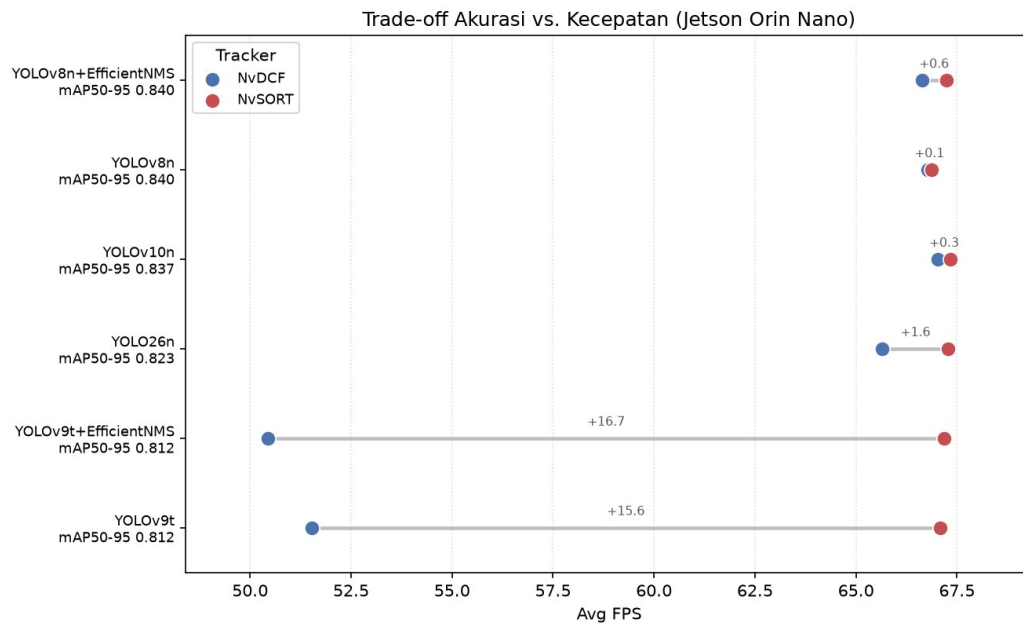


Diagram pasangan (dumbbell) trade-off akurasi mAP50-95 versus rerata FPS: satu baris per model diurutkan menurut akurasi, titik NvDCF dan NvSORT dihubungkan garis yang panjangnya menunjukkan selisih FPS akibat pergantian tracker

Gambar 3.7 Trade-off akurasi vs. daya sistem total, satu baris per model, NvDCF (biru) vs. NvSORT (merah) dihubungkan garis berlabel selisih daya

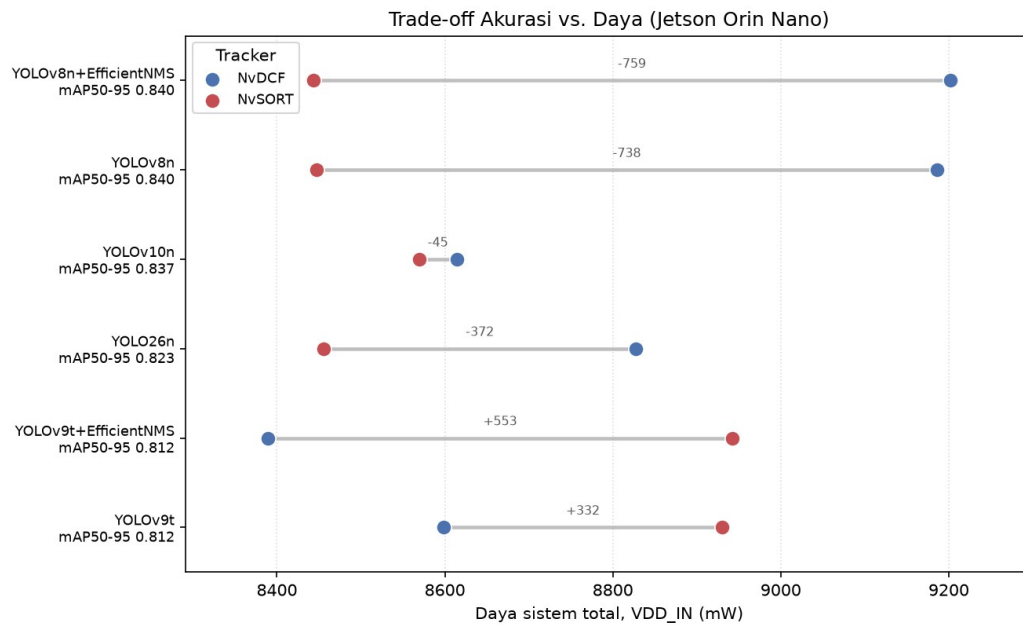


Diagram pasangan (dumbbell) trade-off akurasi mAP50-95 versus daya sistem total: satu baris per model diurutkan menurut akurasi, titik NvDCF dan NvSORT dihubungkan garis yang panjangnya menunjukkan selisih daya akibat pergantian tracker

Kedua diagram di atas menegaskan pengamatan yang mendasari kelima poin rekomendasi berikut: titik NvSORT pada tiap model secara konsisten bergeser ke FPS lebih tinggi (Gambar 3.6, seluruh enam model, selisih +0,1 hingga +16,7 FPS) dan pada empat dari enam model juga bergeser ke daya lebih rendah (Gambar 3.7), kecuali YOLOv9t dan YOLOv9t+EfficientNMS, yang justru mencatat daya *lebih tinggi* pada NvSORT (+332 mW dan +553 mW) meski FPS-nya juga naik paling besar di antara keenam model; sementara jarak vertikal antar-baris model menggambarkan selisih akurasi yang jauh lebih kecil dibanding jarak horizontal antar-tracker pada YOLOv9t, pola yang menjadi dasar rekomendasi berkondisi berikut ini:

42. Prioritas akurasi maksimum: YOLOv8n (mAP50-95 0,8397 tertinggi) dengan tracker NvSORT (FPS 66,86, hampir tidak berbeda dari NvDCF secara statistik, namun latensi tahap *tracking* jauh lebih rendah dan energi/frame lebih hemat).

43. Prioritas efisiensi komputasi/GFLOPs terendah dengan akurasi kompetitif:

YOLO26n (GFLOPs 5,2, mAP50 0,9706 hampir menyamai YOLOv8n) dipasangkan NvSORT, FPS meningkat signifikan (65,66 → 67,29) dan GPU% turun dari 75,6% ke 68,1%.

44. *YOLOv10n sebagai kandidat "default" Pareto-front**: akurasi hampir menyamai YOLOv8n (Δ mAP50-95 hanya 0,3 poin) dengan GFLOPs jauh lebih rendah (6,5 vs. 8,1) dan FPS tertinggi di antara keempat model baseline (67,02–67,34 FPS); efek tracker pada model ini juga paling kecil dan tidak signifikan, membuatnya paling "toleran" terhadap pilihan tracker* apa pun.*

45. *YOLOv9t tidak direkomendasikan pada konfigurasi default (NvDCF): akurasi paling rendah di antara keempat model dan menunjukkan interaksi bottleneck paling parah dengan tracker berat. Jika arsitektur ini tetap ingin dipakai, NvSORT bukan lagi opsional melainkan hampir wajib: kombinasi YOLOv9t+NvDCF adalah satu-satunya yang mendekati (meski masih melampaui) ambang real-time* dengan margin tersempit.*

46. **EfficientNMS tidak direkomendasikan** untuk kedua model yang diuji pada Jetson Orin Nano: tidak memberi keuntungan *throughput* yang signifikan (YOLOv8n) atau secara signifikan lebih lambat (YOLOv9t), sebagaimana dibahas pada subbab dampak EfficientNMS terhadap *throughput*.

mAP50-95 pada tabel/grafik trade-off ini masih memakai proxy FP32; interpretasi di atas akan lebih kuat setelah verifikasi akurasi as-deployed FP16 tersedia, meski deviasi yang diharapkan kecil berdasarkan literatur umum kuantisasi FP16 pada model YOLO.*

Perbandingan dengan penelitian terkait. Temuan hasil negatif EfficientNMS melengkapi, bukan bertentangan dengan, literatur akselerasi NMS yang dirujuk pada bab pendahuluan (Chen dkk., 2022; Oro dkk., 2022; Yang dkk., 2025): ketiga studi tersebut menunjukkan percepatan besar dengan membangun akselerator/kernel *kustom dari nol*, sedangkan penelitian ini menguji pendekatan yang

lebih umum diadopsi pengembang aplikasi, yaitu *plugin* vendor siap pakai (EfficientNMS). Hasil yang berbeda arah ini menegaskan bahwa klaim akselerasi NMS paralel pada literatur **tidak otomatis berlaku umum** untuk semua strategi implementasi, terutama pada model kelas *nano/tiny* yang biaya NMS *baseline*-nya sudah relatif kecil. Sementara itu, untuk RM3, tidak ditemukan studi perbandingan langsung yang mengukur efisiensi komputasi NvDCF vs. NvSORT pada *pipeline* DeepStream di perangkat Jetson-*class*; temuan efisiensi komputasi *tracking* pada bab ini (penghematan NvSORT bersifat universal pada level komponen namun kondisional pada model di level *throughput*) dengan demikian mengisi celah literatur tersebut, sejalan dengan kerangka metodologis MLPerf Mobile Inference Benchmark yang menjadi rujukan pendekatan "akurasi sebagai ambang, komputasi sebagai variabel yang diukur".

3.6.2 Keterbatasan Sistem

Keterbatasan metodologis eksekusi 60 run ini:

47. **Durasi klip video pengujian relatif singkat** ($\pm 13\text{--}20$ detik per *run*, lebih pendek dari rekomendasi 180 detik pada protokol pengujian) karena orkestrasi otomatis tidak menetapkan batas durasi eksplisit dan bergantung pada panjang alami video input. Setelah buang *warm-up* 10 detik, jumlah *frame* yang dianalisis per skenario berkisar 633–1.399 (gabungan 5 repetisi), cukup untuk membedakan pola besar seperti pada perbandingan efisiensi komputasi *tracking*, tetapi estimasi pada skenario dengan n lebih kecil (mis. NvSORT, $\sim 630\text{--}660$ *frame*) memiliki margin ketidakpastian yang lebih besar dibanding jika direkam pada klip yang lebih panjang.
48. **Satu klip video dengan satu tingkat kepadatan objek:** hasil ini belum mengonfirmasi apakah pola *bottleneck* YOLOv9t+NvDCF yang diamati pada dekomposisi latensi dan perbandingan efisiensi *tracking* akan makin parah atau justru mengecil pada skenario lalu lintas yang lebih padat/lebih jarang.

Keterbatasan modul memori 4GB. Seluruh 12 konfigurasi menggunakan 1.265–1.449 MB RAM dari total 4GB yang tersedia pada modul Jetson Orin Nano yang dipakai, sekitar 31–35% kapasitas untuk satu *stream* video dan satu model aktif. Meski masih menyisakan *headroom* untuk skenario satu-kamera satu-model seperti pada penelitian ini, angka ini mengindikasikan bahwa skenario *deployment* ADAS yang lebih kompleks (mis. beberapa kamera sekaligus, atau beberapa model berjalan bersamaan untuk tugas persepsi berbeda) berisiko mendekati batas kapasitas memori pada SKU 4GB, sebuah pertimbangan praktis bagi pengembang yang mempertimbangkan platform ini untuk sistem produksi, di luar SKU 8GB yang tidak diuji pada penelitian ini.

Pengaruh thermal throttling. Risiko thermal throttling dimitigasi secara prosedural melalui jeda cooldown 60 detik antar skenario dan pembersihan tembolok (cache) sistem, mengikuti rekomendasi protokol pengujian. Namun demikian, penelitian ini tidak mengukur suhu SoC secara langsung: kanal pengumpulan data perangkat keras hanya mencakup GPU%, CPU%, RAM, dan daya per-jalur (rail) dari utilitas pemantauan bawaan Jetson Linux, tanpa membaca zona termal sistem secara terpisah. Dengan demikian, efektivitas mitigasi cooldown dalam mencegah throttling bersifat prosedural (mengikuti praktik yang direkomendasikan), bukan terverifikasi langsung dengan data suhu, sebuah keterbatasan instrumentasi yang perlu dicantumkan secara jujur.*

Rekomendasi pengembangan lanjutan (diurutkan dari usaha kecil/dampak sedang ke usaha besar/dampak besar):

49. Menyelesaikan verifikasi akurasi *as-deployed* FP16 sebagai prioritas utama, karena seluruh rekomendasi *trade-off* pada pembahasan akhir bab ini masih bergantung pada *proxy* FP32.
50. Menambah kanal pengukuran suhu SoC (utilitas pemantauan bawaan Jetson Linux sudah melaporkan suhu, namun belum diekstraksi oleh mekanisme

pencatatan yang dipakai saat ini) untuk memverifikasi langsung asumsi mitigasi *thermal throttling* di atas, alih-alih bergantung pada prosedur *cooldown* semata.

51. Eksperimen presisi INT8 sebagai variabel tambahan (di luar SKU tanpa DLA seperti pada penelitian ini), untuk melengkapi perbandingan FP16 vs. INT8 secara terukur, bukan cuma teoretis.
52. Menguji skenario tambahan (kepadatan lalu lintas tinggi, cahaya rendah, cuaca buruk, klip video lebih panjang sesuai rekomendasi protokol) untuk menguji generalisasi temuan performa dan efisiensi *tracking* pada subbab-subbab sebelumnya di luar satu klip video terkontrol yang dipakai penelitian ini.
53. Mengukur kualitas/ketahanan *tracking* (ID *switch*, MOTA/IDF1) secara terpisah dari efisiensi komputasi (di luar batasan lingkup penelitian ini), untuk melengkapi gambaran *trade-off* NvDCF vs. NvSORT secara menyeluruh.
54. Menguji skenario *deployment* multi-kamera/multi-model pada SKU 4GB maupun 8GB untuk mengonfirmasi secara empiris batas praktis keterbatasan memori yang didiskusikan di atas.

BAB IV KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil pengujian dan pembahasan pada BAB III (60 *run*, terdiri atas 6 model $\times$ 2 *tracker* $\times$ 5 repetisi, pada Jetson Orin Nano), kesimpulan penelitian ini disusun untuk menjawab ketiga rumusan masalah dan tujuan penelitian yang ditetapkan pada Bab I sebagai berikut:

55. *Kinerja real-time pipeline Nvidia DeepStream pada Jetson Orin Nano (menjawab RM1/Tujuan 1). Keempat model deteksi baseline (YOLOv8n, YOLOv9t, YOLOv10n, YOLO26n) berhasil berjalan real-time pada Jetson Orin Nano dengan margin yang besar terhadap ambang 30 FPS, dengan rerata FPS berkisar 51,52 (YOLOv9t) hingga 67,02 (YOLOv10n), dan seluruh 60 run individual, tanpa kecuali, memenuhi ambang tersebut, bahkan skenario paling lambat masih melampauinya sebesar 66%. Analisis latensi per-komponen mengidentifikasi YOLOv9t sebagai outlier konsisten: latensi end-to-end-nya (rerata 457,67 ms, rerata P95 536,32 ms) jauh di atas ketiga model lain, disebabkan backpressure pada latensi tahap pra-multiplexing (360,92 ms) akibat interaksi arsitektur PGI/GELAN yang lebih sequential dengan biaya tracker NvDCF di tahap hilir. Ditemukan pula bahwa GFLOPs teoretis tidak berbanding lurus dengan performa aktual di perangkat edge* (YOLO26n memiliki GFLOPs terendah namun bukan model tercepat).*
56. *Pengaruh NMS paralel berbasis TensorRT plugin (EfficientNMS) terhadap efisiensi pipeline (menjawab RM2/Tujuan 2). EfficientNMS tidak terbukti meningkatkan efisiensi pipeline** secara signifikan pada kedua model yang diuji (YOLOv8n, YOLOv9t). Pada tahap inferensi, latensi tahap inferensi membaik, tetapi hanya dalam orde sub-milidetik, konsisten dengan biaya komputasi plugin EfficientNMS itu sendiri yang memang sudah sangat murah (~0,05 ms) pada TensorRT 10.3. Pada tingkat throughput keseluruhan, perbedaan FPS pada*

YOLOv8n tidak signifikan secara statistik ($p = 0,592$), sedangkan pada YOLOv9t perbedaannya justru signifikan namun berlawanan arah dari hipotesis: EfficientNMS 2,1% lebih lambat ($p = 0,023$). Temuan negatif ini bukan kegagalan implementasi, melainkan konsekuensi dari sifat plugin EfficientNMS sebagai tahap akhir (tail) yang tidak overlap dengan komputasi backbone, serta karena latensi tahap inferensi bukan bottleneck dominan pada kelas model nano/tiny ini. Pembahasan model NMS-free (YOLOv10n, YOLO26n) memperkuat simpulan ini: pendekatan menghilangkan NMS secara arsitektural pun tidak otomatis menjamin throughput tertinggi, karena latensi tahap pra-multiplexing dan latensi tahap tracking tetap menjadi kontributor lebih dominan terhadap FPS akhir dibanding latensi tahap inferensi.*

57. *Pengaruh pemilihan algoritma tracking (NvDCF vs. NvSORT) terhadap efisiensi komputasi (menjawab RM3/Tujuan 3). Pada level komponen, biaya komputasi NvDCF pada tahap tracking secara konsisten $\sim 12\times$ – $32\times$ lebih tinggi daripada NvSORT di seluruh enam model yang diuji. Temuan ini bersifat arsitektural dan tidak bergantung pada model deteksi di hulunya, sejalan dengan perbedaan mendasar antara ekstraksi fitur berbasis piksel (NvDCF) dan pendekatan murni berbasis gerak (NvSORT). Namun demikian, penghematan ini bersifat kondisional pada level throughput akhir pipeline: peningkatan FPS yang besar dan sangat signifikan ($+30\%$ dan $+33\%$, $p < 0,0001$) hanya teramati pada YOLOv9t, satu-satunya model yang headroom waktu-per-frame-nya sudah hampir habis, sedangkan pada YOLOv8n, YOLOv10n, dan YOLO26n selisihnya kecil secara praktis ($0,09$ – $1,63$ FPS) karena biaya tambahan NvDCF "tersembunyi" di dalam slack yang masih tersedia. Dari sisi resource, NvSORT konsisten lebih hemat energi per frame di seluruh enam model, meski daya sesaat pada jalur utama (VDD_IN) tidak selalu lebih rendah. Implikasi praktisnya, pemilihan tracker tidak dapat dievaluasi secara independen dari model deteksi yang dipasangkannya: dampaknya terhadap sistem secara keseluruhan hanya*

dapat diketahui melalui pengukuran end-to-end, bukan dari spesifikasi komponen semata.*

Secara keseluruhan, temuan di atas menunjukkan bahwa optimasi *real-time pipeline* Nvidia DeepStream pada Jetson Orin Nano untuk aplikasi ADAS **tidak dapat digeneralisasi dengan satu resep tunggal**, melainkan bergantung pada kombinasi model dan konfigurasi yang dipasangkan. Rekomendasi berkondisi yang disusun pada Bab III merangkum hal ini: YOLOv8n dengan NvSORT untuk prioritas akurasi maksimum, YOLOv26n dengan NvSORT untuk prioritas efisiensi komputasi dengan akurasi kompetitif, YOLOv10n sebagai kandidat *default* Pareto-front yang paling toleran terhadap pilihan *tracker* apa pun, sedangkan YOLOv9t tidak direkomendasikan pada konfigurasi *default* (NvDCF) kecuali dipasangkan dengan NvSORT. Perlu dicatat bahwa nilai mAP50-95 yang mendasari rekomendasi ini masih berupa *proxy* FP32; verifikasi akurasi *as-deployed* FP16 **belum dieksekusi** pada penelitian ini, sehingga kesimpulan di atas, khususnya yang menyangkut trade-off akurasi, masih bersyarat pada asumsi bahwa deviasi kuantisasi FP16 kecil, sebagaimana umum dilaporkan pada literatur model YOLO, namun belum diverifikasi secara independen pada perangkat target penelitian ini.

4.2 Saran

Berdasarkan keterbatasan metodologis dan temuan yang diuraikan pada Bab III, berikut saran yang diajukan bagi penelitian lanjutan maupun pengembangan sistem, diurutkan dari usaha kecil/dampak sedang ke usaha besar/dampak besar:

58. *Menyelesaikan verifikasi akurasi as-deployed FP16 sebagai prioritas utama.*

Infrastruktur pendukungnya, yaitu mekanisme pencatatan luaran deteksi mentah, utilitas konversi kumpulan gambar validasi menjadi video, dan utilitas evaluasi mAP berbasis standar COCO, sudah tersedia, sehingga langkah yang tersisa murni eksekusi lapangan di Jetson. Langkah ini penting karena seluruh

rekomendasi trade-off akurasi pada penelitian ini masih bergantung pada proxy FP32.*

59. **Menambahkan kanal pengukuran suhu SoC** pada mekanisme penguraian (*parsing*) log utilisasi perangkat keras (*tegrastats* sebenarnya sudah melaporkan suhu tetapi belum diekstraksi) untuk memverifikasi secara langsung efektivitas mitigasi *thermal throttling* (jeda *cooldown* 60 detik), yang pada penelitian ini baru bersifat prosedural, bukan terverifikasi dengan data suhu aktual.
60. *Mengeksplorasi jalur optimasi lain di luar plugin EfficientNMS itu sendiri, mengingat temuan negatif pada RM2, mengikuti urutan prioritas berikut: (a) menaikkan ambang skor kepercayaan (score threshold) atau menurunkan jumlah kandidat deteksi, selama evaluasi mAP/recall masih mengizinkan; (b) menguji batas maksimum bounding box keluaran yang lebih kecil (100/200) bila jumlah objek aktual tidak pernah mendekati batas tersebut, untuk mengurangi beban tracker/parser alih-alih selection internal plugin; (c) melakukan profil mendalam dengan perkakas profil TensorRT atau Nsight Systems sebelum menyimpulkan sumber bottleneck secara pasti, ketimbang hanya dari satu kali jalan pipeline; (d) mempertimbangkan arsitektur head NMS-free yang sudah tersedia pada penelitian ini (YOLOv10n, YOLO26n) sebagai alternatif yang lebih menjanjikan daripada menulis kernel NMS generik dari awal; (e) custom plugin baru layak dipertimbangkan bila dapat mem-fuse seluruh tahap decode-threshold-NMS-compact output dalam satu modul plugin* TensorRT khusus untuk konfigurasi target.*
61. **Melakukan eksperimen presisi INT8** sebagai variabel tambahan (di luar SKU tanpa DLA seperti Jetson Orin Nano yang dipakai penelitian ini), untuk melengkapi perbandingan FP16 vs. INT8 secara terukur, bukan sekadar argumen teoretis.
62. **Menguji skenario tambahan**, seperti kepadatan lalu lintas tinggi, kondisi cahaya rendah, cuaca buruk, serta durasi klip video yang lebih panjang sesuai rekomendasi protokol pengujian yang digunakan pada penelitian ini, untuk

menguji generalisasi temuan penelitian ini di luar satu klip video terkontrol yang dipakai.

63. *Mengukur kualitas/ketahanan tracking** (jumlah ID switch, MOTA/IDF1) secara terpisah dari efisiensi komputasi yang dievaluasi pada penelitian ini (di luar scope penelitian ini), agar gambaran trade-off NvDCF vs. NvSORT lebih menyeluruh, mencakup dimensi akurasi tracking, bukan hanya kecepatan dan konsumsi resource*.*
64. *Menguji skenario deployment multi-kamera/multi-model pada SKU 4GB maupun 8GB Jetson Orin Nano, untuk mengonfirmasi secara empiris batas praktis keterbatasan memori yang teridentifikasi pada penelitian ini (penggunaan 31–35% dari kapasitas 4GB untuk satu stream*/satu model) pada konfigurasi yang lebih kompleks dan lebih merepresentasikan kebutuhan ADAS produksi.*
65. *Memanfaatkan kemampuan depth-sensing stereo kamera ZED*, yang pada penelitian ini hanya dipakai sebagai sumber video 2D (sebagaimana disebutkan pada batasan penelitian ini), untuk estimasi jarak kasar ke objek terdeteksi, sebagai langkah lanjutan dari "deteksi 2D" menuju "persepsi 3D" yang lebih dekat dengan kebutuhan fungsional sistem ADAS sesungguhnya.*

Saran-saran di atas disusun agar penelitian lanjutan dapat membangun langsung di atas infrastruktur pengujian (skrip *benchmark*, profil *tracker*, *tooling* NMS) yang sudah tersedia dari penelitian ini, tanpa perlu mengulang tahapan yang sudah selesai divalidasi.

CATATAN VERIFIKASI DAN TINDAK LANJUT PENULIS

Daftar berikut merangkum butir-butir yang masih memerlukan verifikasi, pelengkapan data, atau konfirmasi pembimbing sebelum naskah gabungan ini dapat dianggap final. Daftar ini sengaja dipisahkan dari isi Bab I–IV agar alur akademis pada bab-bab tersebut tetap terbaca sebagai naskah yang sudah tuntas, sambil tetap menjaga transparansi mengenai bagian yang belum diselesaikan (sesuai prinsip non-fabrikasi data pada proyek ini). Naskah ini sudah mencakup keempat bab (Pendahuluan, Metode Penelitian, Hasil dan Pembahasan, serta Kesimpulan dan Saran) berdasarkan data eksekusi 60 run benchmark aktual ($6 \text{ model} \times 2 \text{ tracker} \times 5 \text{ repetisi}$) pada Jetson Orin Nano; satu-satunya bagian hasil yang masih tertunda adalah verifikasi akurasi as-deployed FP16 (lihat butir di bawah).

- **[Prioritas tinggi]** Penggantian rumusan masalah #3 (sumbu Deep Learning Accelerator/DLA menjadi perbandingan efisiensi komputasi algoritma tracking NvDCF vs. NvSORT) pada Bab I belum dikonfirmasi ke dosen pembimbing. Penulis memilih melanjutkan penulisan draf terlebih dahulu hingga Bab IV selesai, dengan rencana merevisi pada bimbingan berikutnya apabila perubahan ini tidak disetujui.
- **[Prioritas tinggi]** Verifikasi akurasi as-deployed FP16 pada pembahasan hasil baru selesai pada tahap infrastruktur kode (probe dump deteksi, konversi video evaluasi, dan skrip penghitungan mAP). Ekspor dataset validasi ke perangkat Jetson, eksekusi pengujian yang sesungguhnya, dan pembaruan angka luaran belum dilakukan. Bab IV sudah mencantumkan keterbatasan ini secara eksplisit, sehingga kesimpulan terkait trade-off akurasi masih bersyarat pada proxy FP32 dan tidak diklaim pasti.
- **[Prioritas sedang]** Identifikasi varian Jetson Orin Nano 4GB pada Bab II diturunkan dari pembacaan mode daya maksimum (`nvpmodel -q`), bukan dari

inspeksi label fisik modul. Verifikasi independen (mis. lewat pembacaan device-tree perangkat) belum dilakukan.

- **[Prioritas sedang]** Versi persis GStreamer, GLib, dan CUDA Toolkit yang terpasang pada perangkat pengujian belum dicatat sebagai metadata reproducibility (mis. lewat `gst-inspect-1.0 --version`, `nvcc --version`, `dpkg -l`).
- **[Prioritas sedang]** Hyperparameter pelatihan model pretrained (rasio split, seed, jumlah epoch, ukuran batch, optimizer, augmentasi, sumber bobot pretrained) belum dipindahkan dari catatan proses pelatihan ke dokumentasi resmi proyek.
- **[Prioritas sedang]** Ambang selisih mAP FP16-vs-FP32 yang dianggap “dapat diabaikan” belum dikunci ke angka final, akan ditentukan berdasarkan referensi literatur setelah data aktual tersedia, untuk menghindari penyesuaian kriteria setelah melihat hasil. Ambang FPS target real-time sudah dikunci ke 30 FPS.
- **[Prioritas rendah]** Nama laboratorium spesifik tempat pelaksanaan pengujian perangkat keras belum dikonfirmasi penulis, belum tercatat secara eksplisit di dokumen proyek mana pun.
- **[Prioritas rendah]** Satu sitasi pada Daftar Pustaka proposal awal, yaitu Wu dkk. (2024, “Road object detection based on improved YOLOv8 for real-time traffic scenarios”, *Sensors* 24(3), 1023), telah dihapus setelah verifikasi DOI melalui Crossref menunjukkan bahwa rujukan tersebut sebenarnya milik artikel lain yang tidak berkaitan. Sitasi ini tidak pernah dipakai sebagai rujukan substantif di bagian mana pun pada naskah ini, namun tetap perlu disampaikan kepada dosen pembimbing karena merupakan bagian dari Daftar Pustaka yang telah disetujui pada seminar proposal.
- **[Prioritas rendah]** Tujuh sitasi pada Bab I (Choi, Nigade, Zhang, Ruiz-Barroso, Suder, Seyfipour, Shah) masih dirangkum setingkat klaim tematik mengikuti proposal, perlu dibaca ulang dari sumber aslinya apabila versi final membutuhkan ringkasan metodologi/hasil yang lebih dalam per jurnal.